

ESWIN

ESSDK快速入门指南

Rev1.1

2026-08-04

声明

知识产权声明

除非另有说明，ESWIN 保留对本产品的所有知识产权。未经 ESWIN 授权，不得对本产品进行反向工程、反编译，不得修改、改编、制作、分发本产品的衍生作品。

ESWIN 保留更新本文档或改进其中所提及的产品或服务的权利。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标，未经授权，不得擅自使用。

责任限制

除本文档或交易文书另有约定外，ESWIN 不提供任何明示、暗示或法律上的保证，包括但不限于适销性、适用性、所有权或非侵权的任何暗示或明示保证，或因任何建议、规格、样品、交易、惯例或贸易惯例而产生的任何保证。

ESWIN 不对任何意外性、惩罚性、间接性的损害承担责任，包括但不限于利润损失、业务中断或采购任何替代商品或服务而产生的成本。

修订记录

版本号	日期	说明
v1.1	2026-08-04	量化编译和部署，根据 20260630 交付物同步调整
v1.0	2026-04-20	重构文档
v0.1	2025-04-14	初始版本

目录

声明 1

 知识产权声明 1

 责任限制 1

修订记录 2

1. 概述 5

 1.1. ESSDK 核心架构 5

 1.1.1. 基础层模块说明 5

 1.1.2. 加速层模块说明 5

 1.2. 文档索引 6

 1.3. 推荐学习路径 6

2. 量化编译与部署 (YOLOv5s) 7

 2.1. 总体流程概览 7

 2.1.1. 前置条件 7

 2.1.2. 涉及工具 7

 2.1.3. 发布包目录结构 8

 2.2. 步骤 1: 模型导出 (PyTorch → ONNX) 9

 2.3. 步骤 2: 模型裁剪 (去除后处理算子) 10

 2.4. 步骤 3: 量化/编译环境搭建 10

 2.4.1. 安装 Docker 10

 2.4.2. 加载 Docker 镜像 11

 2.4.3. 启动 EsQuant Docker 容器 11

 2.4.4. 在容器内安装 EsQuant 11

 2.5. 步骤 4: 模型量化 12

 2.5.1. 准备校准数据集 12

 2.5.2. 编写量化配置文件 13

 2.5.3. 执行量化 16

 2.5.4. 检查量化输出 17

 2.5.5. 精度分析与调优 18

 2.5.6. 生成后处理配置文件 18

 2.6. 步骤 5: 模型编译 19

 2.6.1. 执行编译 19

 2.6.2. 检查编译输出 21

 2.6.3. 准备 model.json 21

 2.7. 步骤 6: 板端部署与推理 21

 2.7.1. 传输文件到开发板 21

 2.7.2. 运行推理 22

 2.7.3. 计算模型精度 24

 2.7.4. 运行性能测试 24

3. 模型运行 26

 3.1. 模型运行概述 26

 3.1.1. 算法运行基本流程 26

 3.1.2. 硬件加速支持情况 26

 3.1.3. 软硬件模块关系图 27

 3.1.4. 核心数据流 27

 3.2. Python 编程详解 28

 3.2.1. 加载模型 28

 3.2.2. 解码 29

 3.2.3. 前处理 31

 3.2.4. 推理 33

 3.2.5. 后处理 34

 3.2.6. 完整示例 (多流流水线) 36

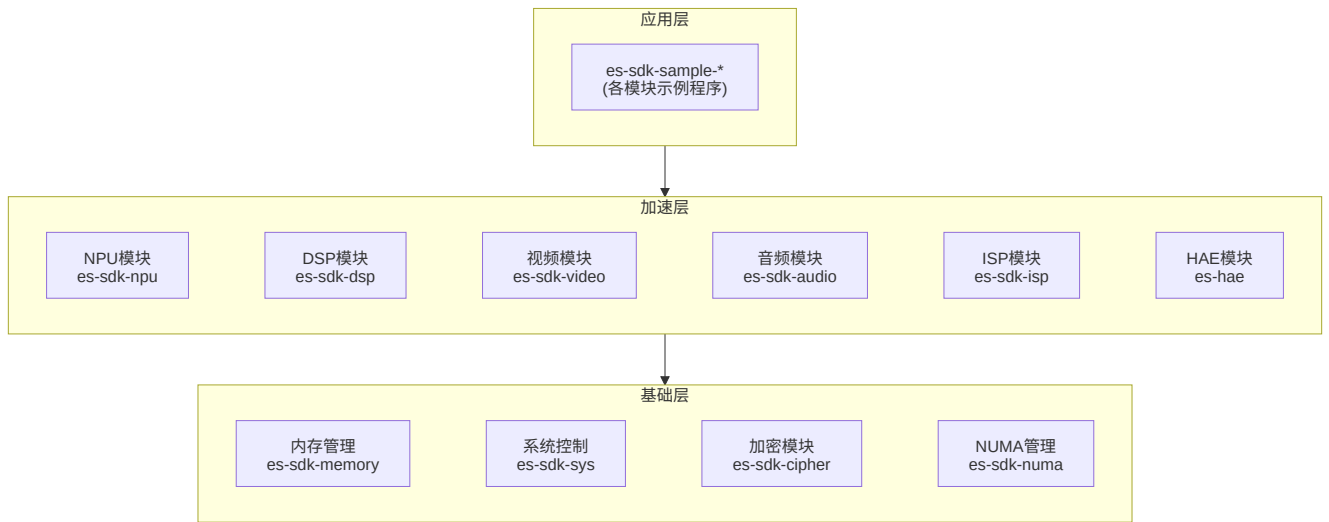
- 3.3. C++编程详解 38
 - 3.3.1. 加载模型 38
 - 3.3.2. 解码 39
 - 3.3.3. 前处理 40
 - 3.3.4. 推理 42
 - 3.3.5. 后处理 44
 - 3.3.6. 完整示例 45
 - 3.3.7. 资源释放 49
- 4. 大模型部署与运行 (DeepSeek) 50
 - 4.1. 获取模型包 50
 - 4.2. 版本兼容性确认 52
 - 4.3. 模型拷贝 52
 - 4.4. 扩充 MMZ 到 11G 52
 - 4.5. 修改模型配置文件 53
 - 4.6. 运行 DeepSeek 模型 54

1. 概述

本文档旨在介绍 ESSDK (Eswin Software Development Kit) 核心架构、功能模块以及学习路径，帮助开发者完成从算法模型到芯片算力的转化与部署，快速上手 EIC77 系列 RISC-V AI SoC 的开发。

1.1. ESSDK 核心架构

ESSDK 是专为 EIC77 系列芯片打造的异构计算软件开发套件。它通过分层设计，打通了从上层应用到底层硬件加速单元（NPU/DSP/VPU）的通路。



1.1.1. 基础层模块说明

基础层为上层应用提供核心的资源管理与系统服务能力。

模块名称	Deb 包	主要功能	典型应用场景
系统控制	es-sdk-sys	系统管理、版本管理、模块间绑定管理。	系统初始化、模块间绑定。
内存管理	es-sdk-memory	提供通用内存和 VB (Video Buffer) 内存的分配管理接口。	视频处理、NPU 推理的内存分配与生命周期管理。
内存复制	es-sdk-memcp	高效异步数据传输和任务调度。	大数据块拷贝加速。
加密模块	es-sdk-cipher	提供 AES、DES、RSA 等加解密算法。	数据安全保护。
NUMA 管理	es-sdk-numa	多 NUMA 节点管理。	EIC7702 中多核调度优化。

1.1.2. 加速层模块说明

加速层是 ESSDK 的核心，提供各类硬件加速能力。

模块名称	Deb 包	主要功能	典型应用场景
NPU 模块	es-sdk-npu	NPU 推理运行时，包含 Runtime 库和固件。	神经网络模型推理。

模块名称	Deb 包	主要功能	典型应用场景
DSP 模块	es-sdk-dsp	DSP 推理运行时，包含 Runtime 库和固件。	自定义算子加速、信号处理。
视频模块	es-sdk-video	视频编解码 (VDEC/VENC)、视频处理 (VPS)、视频输出 (VO)。	视频监控、视频会议。
音频模块	es-sdk-audio	音频输入输出、音频编解码。	语音识别、音频处理。
ISP 模块	es-sdk-isp	图像信号处理。	摄像头图像处理。
HAE 模块	es-hae	图像处理硬件加速引擎。	图像预处理加速。

1.2. 文档索引

文档名称	说明
ESSDK 快速入门指南	介绍 ESSDK 核心架构、功能模块及学习路径。
BSP 快速入门指南	介绍板卡硬件、开发环境搭建、镜像编译、Bootloader、Kernel 及系统开发等基础流程。
ESSDK 安全算法用户手册	安全算法模块的使用说明，涵盖加密接口调用与配置方法。
ESSDK 内存用户手册	内存管理模块说明，指导如何进行通用内存和 VB 缓冲区的分配与管理。
ESSDK 内存拷贝用户手册	内存拷贝模块说明，提供高效异步数据传输和任务调度的接口指南。
ESSDK 系统控制开发者手册	系统控制模块说明，包含系统管理、版本管理及模块间绑定开发的详细指南。
ESSDK 音频用户手册	音频模块使用说明，介绍音频输入输出、编解码功能及插件配置。
ESSDK 音频开发者手册	音频开发指南，深入讲解音频应用开发的 API 接口与编程逻辑。
ESSDK 视频图像处理开发者手册	视频图像处理系统说明，详述视频前处理（VPS）的 API 调用与开发模式。
ESSDK 视频编码开发者手册	视频编码模块 API 详细说明，指导如何利用硬件编码器进行视频压缩。
ESSDK 视频解码开发者手册	视频解码模块 API 详细说明，指导如何利用硬件解码器处理视频流。
ESSDK 视频输出开发者手册	视频输出模块 API 详细说明，涵盖视频显示与输出的开发接口。
ENNP 用户手册	介绍 ENNP 离线介绍 ENNP 离线工具与 API 接口调用，包括环境搭建、模型量化、编译、仿真与调试。
ENNP 开发者手册	DSP Runtime、NPU Runtime 及 AcceleratorKit 的 API 详细说明与开发指南。

1.3. 推荐学习路径

为了帮助您循序渐进地掌握 EIC77 平台开发，建议按照以下顺序进行学习：

第一步：板卡与基础环境

先阅读《BSP 快速入门指南》与对应板卡的《用户手册》，了解开发板的硬件接口、上电、串口连接、系统镜像编辑、烧录、环境搭建。

第二步：ESSDK 快速入门

阅读本文档《ESSDK 快速入门指南》，了解 ESSDK 的整体架构、工具链使用（模型量化→编译→运行）、模型部署与推理流程（Python/C++ API），以及典型场景（如视频分析、大模型部署）。

第三步：模块深入开发

根据您的开发需求，深入阅读[文档索引](#)中对应的模块详细文档，进行具体的 API 调用、参数调优与业务逻辑开发。

按照此路径，您可以从“板卡基础操作”平滑过渡到“AI 模型部署”，最终实现复杂的边缘 AI 应用开发。

2. 量化编译与部署 (YOLOv5s)

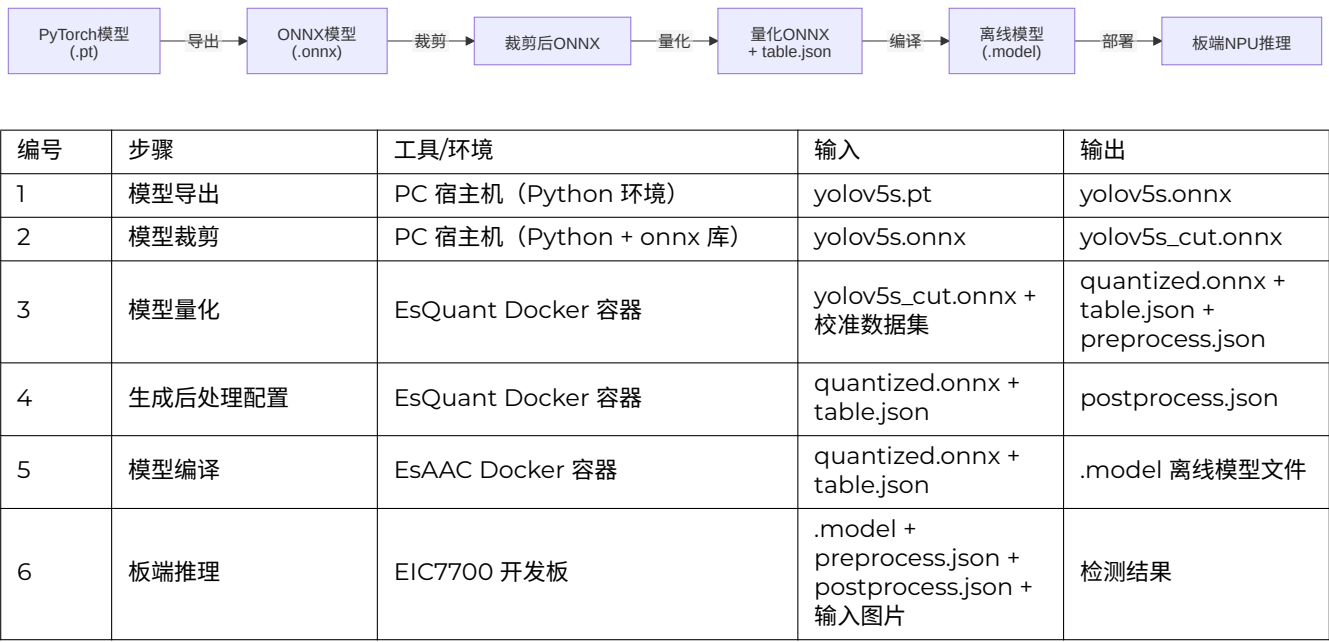
本章以 YOLOv5s 目标检测模型为例，介绍从 PyTorch 训练模型到在 EIC7700 开发板 NPU 上运行推理的完整流程。通过这一典型 CV 模型的落地实践，帮助开发者快速了解 EIC77 芯片模型迁移流程，详细内容参考《ENNP 用户手册》。

提示

若要迁移其他模型，请参考 modelzoo 中的 readme 文件，若 modelzoo 中没有需要的模型，请联系 ESWIN FAE 获取支持。

2.1. 总体流程概览

从算法模型到芯片部署，ESSDK 提供了一套标准化的异构计算工作流，主要分为以下六个步骤：



2.1.1. 前置条件

在开始之前，请确保您已准备好以下资源：

1. 一台安装了 Ubuntu 18.04 以上版本的 x86 主机（运行内存建议 8GB 以上；若量化时使用 GPU 加速，需有 NVIDIA GPU 并安装 CUDA 驱动）
2. ESWIN 提供的两个发布包：
 - **EsNNTools.tar.zst**: NN 工具链包，包含 EsQuant Docker 镜像、EsAAC Docker 镜像、量化脚本、esquant whl 等
 - **modelzoo_vision_*.tar.zst**: 模型库，包含多个预置模型的量化配置、裁剪脚本和参考结果
3. 校准数据集（建议 1000 张图片，推荐从 COCO train2017 中随机选取）。
4. 一块 EIC7700 开发板，用于最终的板端推理验证。

2.1.2. 涉及工具

工具名称	功能说明	所在位置
EsQuant	模型量化工具，支持 INT8/FP16 混合精度量化	esquant_v0.7.3.tar + esquant whl

工具名称	功能说明	所在位置
Example_with_config.py	量化执行脚本，读取配置文件驱动 EsQuant 完成量化流程	640x640/tools/ 目录
EsAAC	模型编译工具，将量化后的 ONNX 编译为 NPU 离线模型	esaac_essimulator_docker.tar 内
EsSimulator	离线模型仿真工具，可在 PC 端模拟 NPU 推理（与 EsAAC 在同一 Docker 中）	同上
sample_npu	板端推理可执行程序，调用 NPU runtime API 执行模型推理	modelzoo sample/bin/ 目录 或 开发板 SDK

2.1.3. 发布包目录结构

先注册 ESWIN 资料中心/开发者论坛账号，用于下载 ESSDK、NN 工具链、模型库和板端软件包。

访问 ESWIN [资料中心（需要注册）](#)

注册并登录后，在资料中心中下载与交付版本匹配的 NN 工具链和模型库资料。不同版本的文件名可能略有差异，本文使用以下名称作为示例。

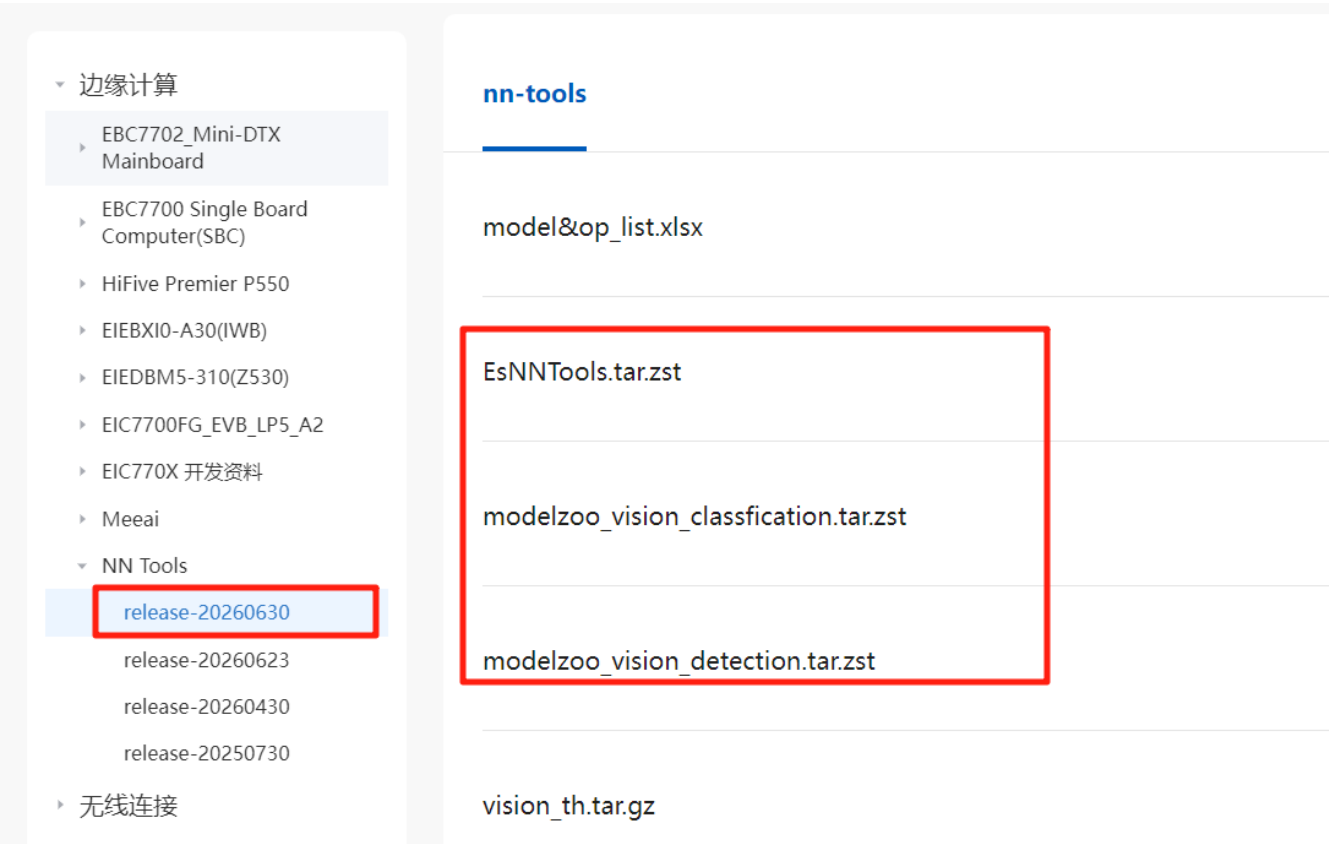


图 1: 工具下载

下载并准备以下资料：

资料	示例文件名	用途
NN 工具链包	EIC7x_Release_xxxxxxx_nn.tar.gz , EsNNTools.tar.zst 或交付版本 NN 包	提供 EsQuant、EsAAC、板端 deb 包、wheel 包
-EsQuant 镜像	esquant_v0.7.3.tar	量化 Docker 镜像
-EsQuant wheel	esquant-1.0.dev0+git.release...whl	容器内安装量化 Python 包
-EsAAC 镜像	esaac_essimulator_docker.tar	编译 Docker 镜像
模型库/视觉模型包	vision/ 或 modelzoo-vision.tar.gz	提供 YOLOv11m 模型目录、脚本和参考产物
板端 SDK 包	es-sdk-dsp_*.deb 、 es-sdk-ak_*.deb	开发板运行环境 (20260630 版本, deb 包已经同步到 apt 源, 所以无额外 deb 包)

可将资料统一放到一个工作目录，并确认关键文件存在，例如：

```
|— esaac_essimulator_docker.tar
|— esquant-1.0.dev0+git.release.2025.730.1a6d7b2f75-py3-none-any.whl
|— esquant_v0.7.3.tar
|— md5sum.txt
|— vision
|   |— classification
|   |— detection
|   |— human_pose_estimation
```

提示

modelzoo 中已包含预先完成量化和编译的模型产物。如果只需使用预置模型，可直接跳到[步骤 6：板端部署与推理](#)。以下步骤演示从头执行完整流程。

以下各步骤，在各个模型文件夹下有 README.ipynb，如果有不一致，以模型文件夹下的 README 为准

2.2. 步骤 1：模型导出（PyTorch → ONNX）

提示

如果您使用 modelzoo 中已提供的 ONNX 模型（位于 pc-models/yolov5/yolov5s/640x640/draft_model/yolov5s.onnx），可跳过此步骤。

模型导出需要在 **宿主机**（非 EsQuant Docker 容器）的 Python 环境中进行，因为需要 PyTorch 和 YOLOv5 的依赖环境。

```
# 在宿主机上操作

# 基于 miniconda 或 venv 创建 Python 环境后，安装导出 ONNX 所需依赖
pip install onnx==1.17.0
pip install onnxruntime==1.21.0
pip install onnxslim==0.1.93
pip install ultralytics==8.4.53
pip install torch==2.6.0
```

```
# 克隆 YOLOv5 仓库
git clone https://github.com/ultralytics/yolov5.git Output/yolov5

# 下载预训练权重
wget -P Output/yolov5 https://github.com/ultralytics/yolov5/releases/download/v7.0/yolov5s.pt

# 导出 640×640 ONNX 模型
cd Output/yolov5
python export.py --weights yolov5s.pt --include onnx --imgsz 640 640 --simplify --opset 12
# 生成文件: yolov5s.onnx
```

2.3. 步骤 2：模型裁剪（去除后处理算子）

YOLOv5 原始 ONNX 模型包含 Detect 后处理层（含 Reshape、Sigmoid 等操作），这些操作会在 NPU 的后处理硬件单元中执行，因此需要将其从 ONNX 模型中裁剪掉，只保留主干网络到三个检测头卷积输出的部分。

modelzoo 中提供了裁剪脚本 `tools/yolov5_crop.py`：

```
import onnx
import sys

if __name__ == "__main__":
    input_names = ["images"]
    output_names = [
        "/model.24/m.0/Conv_output_0",
        "/model.24/m.1/Conv_output_0",
        "/model.24/m.2/Conv_output_0",
    ]
    onnx.utils.extract_model(sys.argv[1], sys.argv[2], input_names, output_names)
```

提示

三个输出节点分别对应 YOLOv5 的三个检测头（大/中/小特征图），这是 YOLOv5 标准结构的固定节点名称。如果您使用自定义模型，需要用 [Netron](#) 可视化工具打开 ONNX 模型，确认需要保留的输出节点名称。

执行裁剪（在 x86 宿主机执行即可，只需安装 onnx 库）：

```
cd /path/to/yolov5/yolov5s/640x640/

# 裁剪 640×640 模型
python tools/yolov5_crop.py ./draft_model/yolov5s.onnx ./verified_model/yolov5s_cut.onnx
```

提示

modelzoo 中 `verified_model/` 目录已包含裁剪好的模型，可直接使用。

2.4. 步骤 3：量化/编译环境搭建

本步骤搭建的是后续模型量化、后处理配置生成和模型编译所需的 Docker 环境。前面的模型导出与模型裁剪基于 x86 宿主机 Python 环境完成，不依赖本步骤的 Docker 环境。

2.4.1. 安装 Docker

在 x86 PC 上安装 Docker：

```
# 安装 Docker
sudo apt-get update
sudo apt-get install docker.io

# 启动并设置开机自启
sudo systemctl start docker
sudo systemctl enable docker

# 配置当前用户免 sudo 执行 docker 命令
sudo groupadd docker      # 若已存在 docker 组会提示，可忽略
sudo usermod -aG docker $USER
sudo service docker restart
newgrp docker

# 验证安装
docker --version
```

2.4.2. 加载 Docker 镜像

从 NN 工具链包中加载两个 Docker 镜像：

```
cd ~/docker_v/EIC7x_Release_20260630_nn/

# 加载 EsQuant 量化工具 Docker 镜像
docker load -i esquant_v0.7.3.tar
# 加载成功后会显示类似：Loaded image: esquant:v0.7.3

# 加载 EsAAC 编译工具 Docker 镜像
docker load -i esaac_essimulator_docker.tar
# 加载成功后会显示类似：Loaded image: esaac_essimulator:latest

# 确认镜像已加载
docker images | grep -E "esquant|esaac"
# 显示镜像列表
# esquant          v0.7.3      ...
# esaac_essimulator latest     ...
```

2.4.3. 启动 EsQuant Docker 容器

```
# 创建并启动容器，将工作目录挂载到容器内
# 这里将 ~/docker_v/ 整体挂载，这样容器内可以同时访问工具链和 modelzoo
docker run --name esquant_work -it --gpus all \
    -v ~/docker_v:/workspace/ \
    esquant:v0.7.3 /bin/bash
```

提示

容器创建后，后续可通过 `docker start esquant_work && docker attach esquant_work` 重新进入。没有 GPU 时，可去掉 `--gpus all`，并在量化配置中将 `device` 改为 `cpu`。

2.4.4. 在容器内安装 EsQuant

进入容器后，安装 EsQuant Python 包及依赖：

```
# 安装 EsQuant whl 包
pip3 install /workspace/EIC7x_Release_20260630_nn/
  ↳ esquant-1.0.[DEV_SUFFIX]+git.[BRANCH].[DEV_SUFFIX2].[GIT_ID_SHORT]-py3-none-any.whl
```

```
root@492824e07ebd:/workspace# pip install /workspace/EIC7x_Release_20250730_nn/esquant-1.0-py3-none-any.whl
Processing ./EIC7x_Release_20250730_nn/esquant-1.0-py3-none-any.whl
Requirement already satisfied: numpy in /opt/conda/lib/python3.10/site-packages (from esquant==1.0) (1.26.4)
Requirement already satisfied: scipy>=1.10.1 in /opt/conda/lib/python3.10/site-packages (from esquant==1.0) (1.10.1)
Requirement already satisfied: onnx>=1.9.0 in /opt/conda/lib/python3.10/site-packages (from esquant==1.0) (1.16.1)
Requirement already satisfied: protobuf in /opt/conda/lib/python3.10/site-packages (from esquant==1.0) (3.20.3)
Requirement already satisfied: opencv-python>=4.10.0.82 in /opt/conda/lib/python3.10/site-packages (from esquant==1.0) (4.10.0.84)
Requirement already satisfied: torch>=1.6.0 in /opt/conda/lib/python3.10/site-packages (from esquant==1.0) (2.3.1)
Requirement already satisfied: torchvision>=0.18.0 in /opt/conda/lib/python3.10/site-packages (from esquant==1.0) (0.18.1)
Requirement already satisfied: tqdm in /opt/conda/lib/python3.10/site-packages (from esquant==1.0) (4.66.4)
Requirement already satisfied: numba in /opt/conda/lib/python3.10/site-packages (from esquant==1.0) (0.60.0)
Requirement already satisfied: filelock in /opt/conda/lib/python3.10/site-packages (from torch>=1.6.0->esquant==1.0) (3.13.1)
Requirement already satisfied: typing-extensions>=4.8.0 in /opt/conda/lib/python3.10/site-packages (from torch>=1.6.0->esquant==1.0) (4.11.0)
Requirement already satisfied: sympy in /opt/conda/lib/python3.10/site-packages (from torch>=1.6.0->esquant==1.0) (1.12.1)
Requirement already satisfied: networkx in /opt/conda/lib/python3.10/site-packages (from torch>=1.6.0->esquant==1.0) (3.2.1)
Requirement already satisfied: Jinja2 in /opt/conda/lib/python3.10/site-packages (from torch>=1.6.0->esquant==1.0) (3.1.4)
Requirement already satisfied: fsspec in /opt/conda/lib/python3.10/site-packages (from torch>=1.6.0->esquant==1.0) (2024.6.0)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /opt/conda/lib/python3.10/site-packages (from torchvision>=0.18.0->esquant==1.0) (10.3.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /opt/conda/lib/python3.10/site-packages (from numba->esquant==1.0) (0.43.0)
Requirement already satisfied: MarkupSafe>=2.0 in /opt/conda/lib/python3.10/site-packages (from Jinja2->torch>=1.6.0->esquant==1.0) (2.1.3)
Requirement already satisfied: mpmath<1.4.0,>=1.1.0 in /opt/conda/lib/python3.10/site-packages (from sympy->torch>=1.6.0->esquant==1.0) (1.3.0)
Installing collected packages: esquant
Successfully installed esquant-1.0
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to u
se a virtual environment instead: https://pip.pypa.io/warnings/venv
```

图 2: 安装 EsQuant whl 包

2.5. 步骤 4: 模型量化

2.5.1. 准备校准数据集

量化需要一组代表性图片作为校准数据，用于统计各层的激活值范围。

数据集要求：

- 建议使用 1000 张图片
- 推荐从 [COCO train2017](#) 中随机选取
- 图片内容应覆盖模型实际推理场景

创建图片列表文件：

```
# 假设 COCO train2017 图片已解压到 /workspace/datasets/train2017/

# 随机选取 1000 张图片，生成列表文件（每行一个绝对路径）
find /workspace/datasets/train2017/ -name "*.jpg" | shuf | head -1000 >
  ↳ /workspace/datasets/cal_images_1000.txt

# 另外选取 1 张图片作为精度分析用
find /workspace/datasets/train2017/ -name "*.jpg" | head -1 > /workspace/datasets/analysis_image_1.txt

# 验证列表文件
wc -l /workspace/datasets/cal_images_1000.txt
# 应输出: 1000
head -3 /workspace/datasets/cal_images_1000.txt
# 应显示类似: /workspace/datasets/train2017/000000xxxxx.jpg
```

```
root@492824e07ebd:/workspace# wc -l /workspace/datasets/cal_images_1000.txt
200 /workspace/datasets/cal_images_1000.txt
root@492824e07ebd:/workspace# head -3 /workspace/datasets/cal_images_1000.txt
/workspace/datasets/train2017/COCO_train2014_000000056289.jpg
/workspace/datasets/train2017/COCO_train2014_000000078963.jpg
/workspace/datasets/train2017/COCO_train2014_000000095924.jpg
```

提示

列表文件中的路径必须是 **Docker 容器内可访问的绝对路径**（以 `/workspace/` 开头），而不是宿主机路径。这是一个常见易错点。

2.5.2. 编写量化配置文件

量化配置文件为 JSON 格式（`esppq_config.json`），定义了模型路径、量化算法和预处理参数。

以 YOLOv5s 640×640 为例，modelzoo 中已提供的原始配置文件 `esppq_config.json` 内容如下：

```
{
  "version": "1.3",
  "model": {
    "model_path": "./verified_model/yolov5s_cut.onnx",
    "save_path": "./quantized_model",
    "images_list": "/IBU_8T/IBU_SOFTWARE/algo_soft/project/quant/quant-table/dataList/det/
    ↪ file_list_1000.txt",
    "analysis_list": "/IBU_8T/IBU_SOFTWARE/algo_soft/project/quant/quant-table/dataList/det/file_list_1.txt"
  },
  "quant": {
    "quantized_method": "per_channel",
    "quantized_dtype": "int8",
    "requant_mode": "mean",
    "quantized_algorithm": "mse_eic",
    "optimization_option": "auto",
    "bias_option": "absmax",
    "nodes_option1": [],
    "nodes_option2": [],
    "nodes_il6": [],
    "output_variables_dtype": {
      "/model.24/m.0/Conv_output_0": "int16",
      "/model.24/m.1/Conv_output_0": "int16",
      "/model.24/m.2/Conv_output_0": "int16"
    },
    "mean": [0, 0, 0],
    "std": [1, 1, 1],
    "norm": true,
    "scale_path": "",
    "enable_analyse": true,
    "device": "cuda"
  },
  "preprocess": {
    "input_format": "RGB",
    "channel_first": true,
    "keep_ratio": true,
    "resize_shape": [640],
    "crop_shape": [640, 640]
  }
}
```

提示

上述配置中的 `images_list` 和 `analysis_list` 路径指向内部开发环境，**不可直接使用**，必须修改为您自己的路径。以下说明具体修改步骤。

2.5.2.1. 修改配置文件（必做）

打开 modelzoo 中已有的配置文件：

```
vi /workspace/vision/detection/pc-models/yolov5/yolov5s/640x640/quantized_model/esppq_config.json
```

需要修改以下两处（其余参数保持默认即可）：

① 修改第 5 行 **images_list**（校准图片列表路径）：

将原始值：

```
"images_list": "/IBU_8T/IBU_SOFTWARE/algo_soft/project/quant/quant-table/dataList/det/file_list_1000.txt",
```

改为您在上文生成的校准图片列表路径：

```
"images_list": "/workspace/datasets/cal_images_1000.txt",
```

② 修改第 6 行 **analysis_list**（精度分析图片列表路径）：

将原始值：

```
"analysis_list": "/IBU_8T/IBU_SOFTWARE/algo_soft/project/quant/quant-table/dataList/det/file_list_1.txt"
```

改为您在上文生成的分析图片列表路径：

```
"analysis_list": "/workspace/datasets/analysis_image_1.txt"
```

提示

路径必须是 **Docker 容器内的绝对路径**（以 `/workspace/` 开头），而不是宿主机路径。这是一个常见的易错点。

2.5.2.2. 可选修改

如果您没有 NVIDIA GPU，还需要修改第 22 行 **device**：

将：

```
"device": "cuda"
```

改为：

```
"device": "cpu"
```

修改完成后，保存文件。修改后的完整配置应如下所示（仅标注了修改行）：

```
{
  "version": "1.3",
  "model": {
    "model_path": "./verified_model/yolov5s_cut.onnx",      ← 根据实际路径调整
    "save_path": "./quantized_model",
    "images_list": "/workspace/datasets/cal_images_1000.txt", ← 修改此行
  }
}
```

```
    "analysis_list": "/workspace/datasets/analysis_image_1.txt" ← 修改此行
  },
  "quant": {
    "quantized_method": "per_channel",
    "quantized_dtype": "int8",
    "requant_mode": "mean",
    "quantized_algorithm": "mse_eic",
    "optimization_option": "auto",
    "bias_option": "absmax",
    "nodes_option1": [],
    "nodes_option2": [],
    "nodes_i16": [],
    "output_variables_dtype": {
      "/model.24/m.0/Conv_output_0": "int16",
      "/model.24/m.1/Conv_output_0": "int16",
      "/model.24/m.2/Conv_output_0": "int16"
    },
    "mean": [0, 0, 0],
    "std": [1, 1, 1],
    "norm": true,
    "scale_path": "",
    "enable_analyse": true,
    "device": "cuda" ← 无 GPU 改为"cpu"
  },
  "preprocess": {
    "input_format": "RGB",
    "channel_first": true,
    "keep_ratio": true,
    "resize_shape": [640],
    "crop_shape": [640, 640]
  }
}
```

关键参数说明：

参数	说明	推荐值/示例
model.model_path	裁剪后的 ONNX 模型路径	./verified_model/yolov5s_cut.onnx
model.save_path	量化结果输出目录	./quantized_model
model.images_list	校准图片列表文件路径（每行一个图片绝对路径）	需指向实际路径
model.analysis_list	精度分析用图片列表（1 张即可）	需指向实际路径
quant.quantized_dtype	量化位宽	int8（默认）
quant.quantized_algorithm	量化校准算法	YOLO 系列用 mse_eic；分类模型用 at_eic
quant.optimization_option	优化策略	auto（自动选择）
quant.output_variables_dtype	指定特定输出节点的数据类型	YOLOv5 三个检测头建议设为 int16 以提升精度
quant.nodes_i16	指定使用 INT16 的中间层列表	[]（默认为空，精度不足时按需添加）
quant.enable_analyse	是否开启精度分析	true
quant.device	量化计算设备	有 GPU 用 cuda，否则用 cpu
preprocess.input_format	输入颜色格式	YOLOv5 用 RGB

参数	说明	推荐值/示例
<code>preprocess.keep_ratio</code>	resize 时是否保持长宽比 (letterbox)	YOLO 系列为 true
<code>preprocess.resize_shape</code>	resize 目标尺寸	[640] (640×640) 或 [416] (416×416)
<code>preprocess.crop_shape</code>	输入张量的 HW 尺寸	[640, 640] 或 [416, 416]

2.5.3. 执行量化

在 EsQuant Docker 容器内执行：

```
cd /path/to/yolov5/yolov5s/640x640/

# 执行量化（以 640×640 为例）
python tools/Example_with_config.py \
    --config_path quantized_model/esppq_config.json \
    --preprocess_name Yolo
```

```
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s# python /workspace/EIC7x_Release_20250730_nn/Example_with_config.py \
--config_path ./640x640/quantized-model/esppq_config.json \
--preprocess_name Yolo
Read SHA from /opt/conda/lib/python3.10/site-packages/esquant/core/sha.txt
No CUDA runtime is found, using CUDA_HOME='/usr/local/cuda'

ESQUANT

[Info] Redundant Cast operations removed.
[Info] No Matmul-Split pattern was found.
[Info] No valid Transpose which can be decomposed was found.
[Info] No suitable matmul + add was found.
[Info] No Add-Reshape-Add-Reshape pattern was found.
[Info] No LayerNorm pattern was found.
[Info] No Gelu pattern was found.
[Info] No quick Gelu pattern was found.
[Info] No suitable matmul + add was found.
[Info] No suitable LayerNormalization+Gemm/ESQUANTBiasFusedMatMul/MatMul was found.
[Info] No valid pattern found for CrossShapedWindow fusion.
[Info] No valid pattern found for InvCrossShapedWindow fusion.
[Info] No valid pattern found for deit to split qkv-matmul fusion.
[Info] No valid pattern found for EVA02 to split qkv-matmul fusion.
[Info] No valid Requant - Add pattern was found.
[Info] No valid pattern found for swint to split qkv-matmul fusion.
[Warning] More than 1 pattern root was found, Complex Pattern might cause memory overflow ...
[07:09:58] ESQUANT Quantize Simplify Pass Running ... Finished.
[07:09:58] ESQUANT Quantization Fusion Pass Running ... Finished.
[07:09:58] ESQUANT Mix Precision Pass Running ... Finished.
Calibration Progress(Phase 1): 100% | 200/200 [00:17<00:00, 11.19it/s]
Calibration Progress(Phase 2): 100% | 200/200 [04:03<00:00, 1.22s/it]

Finished.
[07:16:14] ESQUANT Softmax Scale Optimizer Pass Running ... [Info]
=====
[Info]
=====
Finished.
[07:16:14] ESQUANT Parameter Quantization Pass Running ... Finished.
[07:16:14] EIC Parameter Quantization Pass Running ... Finished.
[07:16:14] ESQUANT Passive Parameter Quantization Running ... Finished.
[07:16:14] ESQUANT Quantization Alignment Pass Running ... Finished.
```

```
/model.2/cv3/conv/Conv: | 99.999%
/model.8/m/m.0/cv1/conv/Conv: | 99.999%
/model.8/cv1/act/Mul: | 99.999%
/model.2/m/m.0/cv1/act/Mul: | 99.999%
/model.4/cv3/conv/Conv: | 99.999%
/model.3/conv/Conv: | 99.999%
/model.2/cv3/act/Mul: | 99.998%
/model.2/cv2/act/Mul: | 99.998%
/model.2/cv2/conv/Conv: | 99.998%
/model.0/conv/Conv: | 99.997%
/model.2/Concat: | 99.997%
/model.2/m/m.0/cv2/act/Mul: | 99.995%
/model.2/m/m.0/cv1/conv/Conv: | 99.995%
/model.2/m/m.0/cv2/conv/Conv: | 99.993%
/model.2/cv1/conv/Conv: | 99.993%
/model.1/conv/Conv: | 99.992%
/model.1/act/Mul: | 99.980%
/model.2/cv1/act/Mul: | 99.972%
[Warning] File ./640x640/quantized-model/quantized.onnx is already existed, Exporter will overwrite it.
[Warning] File ./640x640/quantized-model/table.json is already existed, Exporter will overwrite it.
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s#
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s#
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s#
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s#
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s# ls -alh ./640x640/quantized-model/quantized.onnx
-rw-r--r-- 1 4495 10002 28M Apr 16 07:16 ./640x640/quantized-model/quantized.onnx
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s# ls -alh ./640x640/quantized-model/table.json
-rw-r--r-- 1 4495 10002 114K Apr 16 07:16 ./640x640/quantized-model/table.json
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s# date
Thu Apr 16 07:17:23 UTC 2026
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s# ls -alh 640x640/quantized-model/
total 28M
drwxr-xr-x 2 4495 10002 4.0K Apr 16 07:06 .
drwxr-xr-x 5 4495 10002 87 Apr 16 06:20 ..
-rw-r--r-- 1 4495 10002 1.3K Apr 16 07:06 esppq_config.json
-rw-r--r-- 1 4495 10002 973 Apr 16 06:20 postprocess.json
-rw-r--r-- 1 4495 10002 11K Apr 16 07:16 precision_accumulate_result.txt
-rw-r--r-- 1 4495 10002 11K Apr 16 07:16 precision_reset_result.txt
-rw-r--r-- 1 4495 10002 416 Apr 16 07:16 preprocess.json
-rw-r--r-- 1 4495 10002 28M Apr 16 07:16 quantized.onnx
-rw-r--r-- 1 4495 10002 114K Apr 16 07:16 table.json
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s#
```

参数说明：

- `--config_path`：指向量化配置文件的路径
- `--preprocess_name`：预处理类型。YOLO 系列模型固定使用 `Yolo`（执行 letterbox 缩放 + padding）；分类模型使用 `ImagenetOpencv`。

量化过程会依次执行：加载模型 → 校准数据预处理 → 逐层量化 → 精度分析 → 导出结果。根据校准图片数量和 GPU 性能，耗时约 10~30 分钟。

2.5.4. 检查量化输出

量化成功后，在 `save_path` 指定的目录下生成以下文件：

quantized_model/

- ├── quantized.onnx # 量化后的 ONNX 模型（编译时的输入）
- ├── table.json # 量化参数表（编译时需要）
- ├── preprocess.json # 预处理配置（板端推理时需要）
- ├── precision_accumulate_result.txt # 累计误差分析报告
- └── precision_reset_result.txt # 逐层重置误差分析报告

各文件用途：

文件	用途	后续步骤
quantized.onnx	量化后的模型，作为 EsAAC 编译的输入	模型编译
table.json	量化参数统计表，包含各层的 scale/step 信息	模型编译

文件	用途	后续步骤
preprocess.json	预处理配置（含 mean/std/scale/padding 等参数）	板端推理
precision_accumulate_result.txt	累计误差报告，用于评估量化精度	精度验证
precision_reset_result.txt	逐层重置误差报告，用于定位精度瓶颈层	精度调优

2.5.5. 精度分析与调优

查看累计误差分析报告，检查量化精度是否达标：

```
# 查看最后几层的累计余弦相似度
tail -5 ./quantized_model/precision_accumulate_result.txt
```

输出示例（正常结果）：

```
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s# tail -5 ./640x640/quantized_model/precision_accumulate_result.txt
model.23/cv3/act/Sigmoid      : 0.9863182902336121
model.23/cv3/act/Mul         : 0.953834056854248
model.24/m.2/Conv            : 0.9984406232833862
model.24/m.1/Conv            : 0.9969298839569092
model.24/m.0/Conv            : 0.99727463722229
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s#
```

图 3: 量化精度

判断标准：最后一层（即检测头输出层）的余弦相似度应 ≥ 0.95 。YOLOv5s 标准量化结果通常在 0.997 以上。

如果精度不达标的调优方法：

1. **指定敏感层使用 INT16：**查看 precision_reset_result.txt，找到余弦相似度最低的层，将其层名添加到配置文件的 nodes_i16 列表中：

```
"nodes_i16":["/model.2/cv1/conv/Conv", "/model.4/cv1/conv/Conv"]
```

2. **调整量化算法：**将 quantized_algorithm 从 mse_eic 改为 at_eic，或反之尝试。
3. **检查校准数据集：**确保校准图片与实际推理场景一致，且数量足够（建议 ≥ 500 张）。
4. **确认预处理参数：**mean、std、input_format 等必须与模型训练时的预处理完全一致。

2.5.6. 生成后处理配置文件

后处理配置文件 postprocess.json 包含 NMS 参数、anchor 信息、类别数等，供板端硬件后处理单元使用。modelzoo 提供了自动生成脚本：

```
cd /workspace/vision/detection/pc-models/yolov5/yolov5s/640x640/

# 生成 640*640 的 postprocess.json
python tools/postprocess_gen.py --version yolov5 --path quantized_model
```

提示

该脚本会读取 quantized.onnx、table.json 和 esppq_config.json，自动计算 input_scale、input_shape 等参数并写入 postprocess.json。

生成的 `postprocess.json` 示例 (640×640):

```
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s# python postprocess_gen.py --version yolov5 --path ./640x640/quantized-model/
Jump postprocess.json success to ./640x640/quantized-model/postprocess.json
root@492824e07ebd:/workspace/vision/detection/pc-models/yolov5/yolov5s# cat ./640x640/quantized-model/postprocess.json
{
  "config_version": "1.0.0",
  "kernel_name": "detection_out",
  "detection_net": "yolov5",
  "in_tensor_num": 3,
  "out_tensor_num": 2,
  "input_shape": "[1,255,20,20],[1,255,40,40],[1,255,80,80]",
  "output_shape": "[1,7,1,1024],[1,1,1,1]",
  "input_data_type": "S16,S16,S16",
  "output_data_type": "F32,I32",
  "anchors_num": 3,
  "anchor_scale_table": "[[116, 90, 156, 198, 373, 326],[30, 61, 62, 45, 59, 119],[10, 13, 16, 30, 33, 23]]",
  "cls_num": 80,
  "img_h": 640,
  "img_w": 640,
  "input_scale": "[0.000584765977691859,0.0005253022536635399,0.0006663458188995719]",
  "nms_method": "hard_nms",
  "nms_multi_label": "false",
  "iou_method": "iou",
  "out_box_type": "xminyminxmaxymax",
  "coord_norm_flag": "false",
  "max_boxes_per_class": 200,
  "max_boxes_per_batch": 500,
  "score_threshold": 0.3,
  "iou_threshold": 0.45,
  "soft_nms_sigma": 0.6,
  "effec_img_offset_y": 0,
  "effec_img_offset_x": 0
}
```

图 4: 后处理参数

2.6. 步骤 5: 模型编译

使用 EsAAC 编译器将量化后的 ONNX 模型编译为可在 NPU 上运行的离线模型 (`.model` 文件)。

2.6.1. 执行编译

编译在 EsAAC Docker 容器中执行。以下使用 `docker run` 一次性完成:

```
# 确保在宿主机上执行

cd /path/to/yolov5/yolov5s/640x640/
sudo chmod -R 777 quantized_model

# 编译 640×640 量化模型
docker run --rm --privileged -it \
-v /path/to/640x640/quantized_model:/home/eswin/quantized_model \
-u root -w "/home/eswin" esaac_essimulator:latest bash -c "/home/eswin/EsaAAC \
--input-model ./quantized_model/quantized.onnx \
--output-dir ./quantized_model/ \
--loadable-name yolov5s_640x640 \
--quant-stats ./quantized_model/table.json \
--max-lifetime-threshold 30 \
--enable-split-matmul \
--remap-F0 \
--mapping-cache /home/eswin/quantized_model/yolov5s_640x640_npu_mapper_result.json \
--enable-parallel-m0 \
--split-lutdsp \
--disable-dilation-conv-weight"
```

```
e0004495@ibus01:~/docker_v/vision/detection/es-hw-models/yolov5/yolov5s$ sudo chmod -R 777 models/640x640
[sudo] password for e0004495:
Sorry, try again.
[sudo] password for e0004495:
e0004495@ibus01:~/docker_v/vision/detection/es-hw-models/yolov5/yolov5s$ docker run --rm --privileged -it -v ~/docker_v/~/workspace/ esaac_essimul
ator:20250730 bash -c "-/EsAAC \
--input-model /workspace/vision/detection/pc-models/yolov5/yolov5s/640x640/quantized-model/quantized.onnx \
--output-dir /workspace/vision/detection/es-hw-models/yolov5/yolov5s/models/640x640 \
--loadable-name git_yolov5s_640_mix_1x3x640x640_dyn \
--input-nodes images \
--input-shapes 1,3,640,640 \
--run-device npu \
--quant-stats /workspace/vision/detection/pc-models/yolov5/yolov5s/640x640/quantized-model/table.json"
[EsAAC] Model compiling for npu with shape:1,3,640,640
[EsAAC] Model frontend parsing time: 467ms:423us
DSP_COMPILER [INFO]: LUT_ACT_V2_CONFIG_S size is 0x90
DSP_COMPILER [INFO]:loop_count is: 35
DSP_COMPILER [INFO]:tile_size is: 1296
DSP_COMPILER [INFO]:tile_num is: 37
DSP_COMPILER [INFO]:tail_num is: 22
DSP_COMPILER [INFO]:Copy the lut table directly from memory.
DSP_COMPILER [INFO]:emit kernel library name is lut_act_v2_s8s8
DSP_COMPILER [INFO]: LUT_ACT_V2_CONFIG_S size is 0x90
DSP_COMPILER [INFO]:loop_count is: 18
```

图 5: 编译开始

```
DSP_COMPILER [INFO]: LUT_ACT_V2_CONFIG_S size is 0x90
DSP_COMPILER [INFO]:loop_count is: 13
DSP_COMPILER [INFO]:tile_size is: 640
DSP_COMPILER [INFO]:tile_num is: 25
DSP_COMPILER [INFO]:tail_num is: 20
DSP_COMPILER [INFO]:Copy the lut table directly from memory.
DSP_COMPILER [INFO]:emit kernel library name is lut_act_v2_s8s8
fatal: not a git repository (or any of the parent directories): .git
[EsAAC] Model backend processing for npu with shape:1,3,640,640 time:18s:67ms:44us
[EsAAC] Model compilation completed total time: 18s:816ms:206us
e0004495@ibus01:~/docker_v/vision/detection/es-hw-models/yolov5/yolov5s$ ls -alh models/640x640
total 44M
drwxrwxrwx 6 e0004495 ibu  4.0K Apr 16 16:22 .
drwxr-xr-x 5 e0004495 ibu   68 Apr 16 16:15 ..
drwxrwxrwx 3 eswin  eswin  34 Apr 16 16:19 .git
-rwxrwxrwx 1 e0004495 ibu   13K Apr 16 14:20 es_yolov5s_classes.txt
-rwxrwxrwx 1 e0004495 ibu   9.8M Apr 16 16:23 git_yolov5s_640_mix_1x3x640x640_dyn_npu_b1.model
-rwxrwxrwx 1 eswin  eswin   96 Apr 16 16:23 git_yolov5s_640_mix_1x3x640x640_dyn_npu_b1.ofmap_order.txt
drwxrwxrwx 2 eswin  eswin   4K Apr 16 16:23 postprocess
-rwxrwxrwx 1 e0004495 ibu  167 Apr 16 16:23 model.json
drwxrwxrwx 3 e0004495 ibu   28 Apr 16 14:20 preprocess
-rwxrwxrwx 1 e0004495 ibu   973 Apr 16 14:20 postprocess.json
-rwxrwxrwx 1 e0004495 ibu   34M Apr 16 14:20 prediction.json
-rwxrwxrwx 1 e0004495 ibu   416 Apr 16 14:20 preprocess.json
drwxrwxrwx 2 e0004495 ibu   4.0K Apr 16 14:20 results
e0004495@ibus01:~/docker_v/vision/detection/es-hw-models/yolov5/yolov5s$
```

图 6: 编译完成

EsAAC 参数说明:

参数	说明	示例值
-input-model	量化后的 ONNX 模型路径	quantized.onnx 的完整路径
-output-dir	编译产物输出目录	./quantized_model/
-loadable-name	生成的 .model 文件名前缀	yolov5s_640x640
-quant-stats	量化参数表路径	table.json 的完整路径

提示

编译 416×416 模型时，进入 416x416 目录并将 --loadable-name 改为对应名称即可。

2.6.2. 检查编译输出

编译成功后，输出目录下会生成 .model 文件：

```
ls -lh /path/to/yolov5/yolov5s/640x640/quantized_model/*.model
# 示例输出：
# -rw-r--r-- 1 root root 9.9M ... yolov5s_640x640_npu_b1.model
```

文件名格式为 {loadable-name}_{run-device}_b{batch}.model。

2.6.3. 准备 model.json

板端推理程序需要 model.json 来描述模型信息。新版 modelzoo 的 quantized_model/ 目录已提供 model.json；如果重新编译后文件名发生变化，需要确认 model_file 与实际生成的 .model 文件名一致：

```
{
  "model_list": [
    {
      "model_device": 1,
      "model_file": "yolov5s_640x640_npu_b1.model",
      "model_type": "npu"
    }
  ]
}
```

编译完成后，quantized_model/ 目录应包含以下文件（板端推理所需的最小文件集）：

```
/path/to/yolov5/yolov5s/640x640/quantized_model/
├── yolov5s_640x640_npu_b1.model # 离线模型
├── model.json # 模型描述
├── preprocess.json # 预处理配置
├── postprocess.json # 后处理配置
└── es_yolov5s_classes.txt # 类别标签文件（可选，用于结果显示）
```

2.7. 步骤 6：板端部署与推理**2.7.1. 传输文件到开发板**

将编译产物和推理程序传输到 EIC7700 开发板：

```
# 在 PC 端执行
# 查看开发板 ip: 串口登录后, ip addr show
# debian 开发板用户名, 密码, 默认都是 eswin
cd /path/to/yolov5/yolov5s/640x640/
ssh eswin@<开发板 IP> "mkdir -p /home/eswin/tools"
scp sample/bin/sample_npu eswin@<开发板 IP>:/home/eswin/
scp -r quantized_model eswin@<开发板 IP>:/home/eswin/
scp -r tools/pictures eswin@<开发板 IP>:/home/eswin/
scp tools/map_coco.py eswin@<开发板 IP>:/home/eswin/tools/
```

传输后，开发板上的目录结构：

```
/home/eswin/
├── sample_npu      # 推理可执行程序
├── quantized_model/
│   ├── *.model
│   ├── model.json
│   ├── preprocess.json
│   ├── postprocess.json
│   └── es_yolov5s_classes.txt
├── pictures/
│   └── input0/      # 测试图片
├── tools/
└── map_coco.py
```

2.7.2. 运行推理

SSH 登录开发板，执行推理：

```
# 登录开发板
ssh eswin@<开发板 IP>

sudo apt update
# 触发 sample 依赖的底层驱动 (npu-sdk, vdec-sdk 等) 的自动安装，
sudo apt install es-sdk-sample-npu-runtime

# 必须重启开发板，否则可能报错 deviceid(0)>deviceNum(0)
reboot

# 重启完成后重新登录开发板
ssh eswin@<开发板 IP>

# 创建输出目录
rm -rf ~/output && mkdir -p ~/output

# 运行推理 (640×640 模型，同步模式)
./sample_npu -s 1 -m quantized_model/ -i pictures/ -o output/
```

```
eswin@rockos-eswin:~$ ~/yolov5s/bin/sample_npu -s 1 -m ~/yolov5s/models/640x640/ -i ~/yolov5s/models/640x640/pictures/ -o ~/output
/home/eswin/yolov5s/models/640x640/pictures/input0/000000006818.jpg: box count(1)
class_id: 61.000000( ('toilet')), score: 0.649414, x_min: 292.500000, y_min: 473.750000, x_max: 397.000000, y_max: 523.500000
/home/eswin/yolov5s/models/640x640/pictures/input0/0000000016228.jpg: box count(14)
class_id: 17.000000( ('horse')), score: 0.906250, x_min: 324.500000, y_min: 253.000000, x_max: 534.500000, y_max: 451.500000
class_id: 13.000000( ('bench')), score: 0.714844, x_min: 523.000000, y_min: 333.500000, x_max: 624.000000, y_max: 389.500000
class_id: 0.000000( ('person')), score: 0.655762, x_min: 546.000000, y_min: 264.750000, x_max: 571.000000, y_max: 295.000000
class_id: 0.000000( ('person')), score: 0.568848, x_min: 66.000000, y_min: 302.500000, x_max: 80.875000, y_max: 361.500000
class_id: 0.000000( ('person')), score: 0.567383, x_min: 36.218750, y_min: 304.500000, x_max: 52.187500, y_max: 363.000000
class_id: 0.000000( ('person')), score: 0.553711, x_min: 200.000000, y_min: 289.750000, x_max: 239.250000, y_max: 372.250000
class_id: 25.000000( ('umbrella')), score: 0.551758, x_min: 349.750000, y_min: 232.000000, x_max: 476.250000, y_max: 268.000000
class_id: 0.000000( ('person')), score: 0.503418, x_min: 527.000000, y_min: 267.750000, x_max: 545.500000, y_max: 299.000000
class_id: 0.000000( ('person')), score: 0.473633, x_min: 74.187500, y_min: 206.250000, x_max: 89.500000, y_max: 263.750000
```

图 7: 推理开始


```
class_id: 24.000000( ('backpack')), score: 0.310303, x_min: 343.000000, y_min: 369.000000, x_max: 427.250000, y_max: 441.000000
ASSERT: software/source/common/input_queue.c, InputQueueReturnBuffer, 400
/home/eswin/yolov5s/models/640x640/pictures/input0/000000041888.jpg: box count(3)
class_id: 14.000000( ('bird')), score: 0.845215, x_min: 197.750000, y_min: 285.000000, x_max: 328.500000, y_max: 393.500000
class_id: 18.000000( ('sheep')), score: 0.768555, x_min: 404.750000, y_min: 335.000000, x_max: 532.500000, y_max: 441.500000
class_id: 18.000000( ('sheep')), score: 0.741211, x_min: 293.250000, y_min: 324.500000, x_max: 387.000000, y_max: 432.500000
/home/eswin/yolov5s/models/640x640/pictures/input0/000000058636.jpg: box count(0)
/home/eswin/yolov5s/models/640x640/pictures/input0/000000063154.jpg: box count(3)
class_id: 0.000000( ('person')), score: 0.699219, x_min: 346.500000, y_min: 245.250000, x_max: 389.500000, y_max: 278.250000
class_id: 37.000000( ('surfboard')), score: 0.477539, x_min: 242.250000, y_min: 288.250000, x_max: 354.500000, y_max: 335.000000
class_id: 0.000000( ('person')), score: 0.459229, x_min: 307.000000, y_min: 243.750000, x_max: 389.750000, y_max: 311.250000
eswin@rockos-eswin:~$ ls -alh output/
total 1.3M
drwxrwxr-x 2 eswin eswin 4.0K Apr 16 17:42 .
drwx----- 4 eswin eswin 4.0K Apr 16 17:29 ..
-rw-rw-r-- 1 eswin eswin 6.3K Apr 16 17:42 prediction.json
-rw-rw-r-- 1 eswin eswin  92K Apr 16 17:41 result_000000006818.jpg
-rw-rw-r-- 1 eswin eswin 154K Apr 16 17:41 result_000000016228.jpg
-rw-rw-r-- 1 eswin eswin 150K Apr 16 17:42 result_000000017627.jpg
-rw-rw-r-- 1 eswin eswin 102K Apr 16 17:42 result_000000025560.jpg
-rw-rw-r-- 1 eswin eswin 117K Apr 16 17:42 result_000000035197.jpg
-rw-rw-r-- 1 eswin eswin  39K Apr 16 17:42 result_000000037777.jpg
-rw-rw-r-- 1 eswin eswin 122K Apr 16 17:42 result_000000039956.jpg
-rw-rw-r-- 1 eswin eswin 191K Apr 16 17:42 result_000000041888.jpg
-rw-rw-r-- 1 eswin eswin 154K Apr 16 17:42 result_000000058636.jpg
-rw-rw-r-- 1 eswin eswin 130K Apr 16 17:42 result_000000063154.jpg
eswin@rockos-eswin:~$
```

图 8: 推理完成

sample_npu 参数说明:

参数	说明	示例值
-s	推理模式 (1=同步, 2=异步, 3=多 Context, 4=多 Stream)	1
-m	模型目录路径 (包含 .model、model.json、pre/postprocess.json)	quantized_model/
-i	输入图片目录路径 (图片需放在 input0/ 子目录下)	pictures/
-o	结果输出目录	output/

提示

输入图片需要放在 `-i` 指定目录下的 `input0/` 子目录中。modelzoo 中已按此结构组织。

输出目录 `output/` 中会生成带有检测框的结果图片，可拷贝到有图形界面的系统中预览。

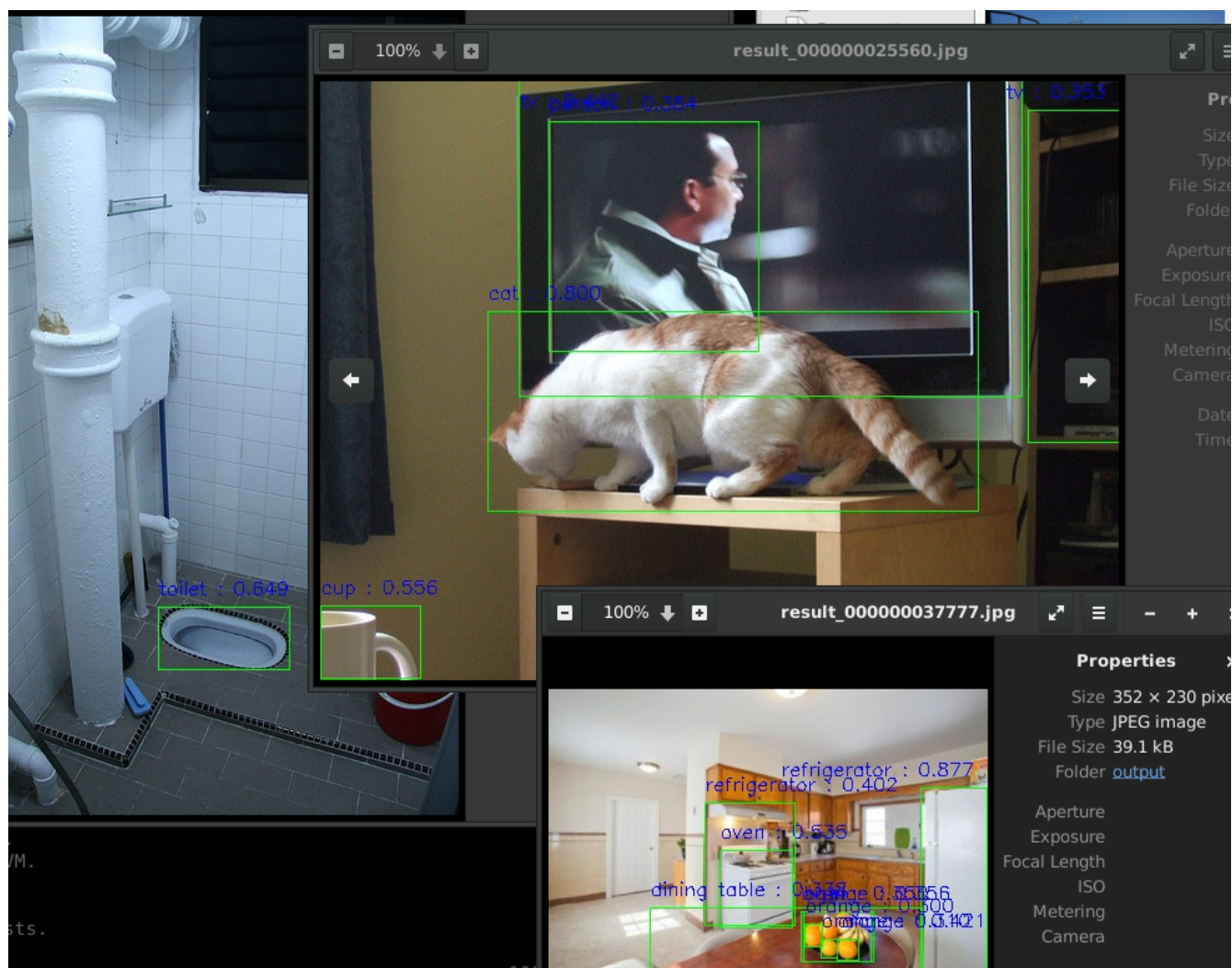


图 9: 检测结果

2.7.3. 计算模型精度

模型精度测试前, 需要将 `postprocess.json` 中的 `score_threshold` 修改为 0.01, `iou_thres` 修改为 0.6。从 <https://images.cocodataset.org/zips/val2017.zip> 下载 COCO val2017 数据集后, 在开发板上基于该数据集运行:

```
rm -rf pictures/input0 && mkdir -p pictures/input0
find val2017 -name "*.jpg" -exec cp {} pictures/input0/ \;
rm -rf output && mkdir -p output
./sample_npu -s 1 -m quantized_model/ -i pictures/ -o output/
```

输出结果 `output/prediction.json` 后, 结合 `instances_val2017.json` (下载自 https://images.cocodataset.org/annotations/annotations_trainval2017.zip), 使用 `map_coco.py` 计算精度:

```
python tools/map_coco.py --groundtruth instances_val2017.json --prediction output/prediction.json
```

2.7.4. 运行性能测试

运行端到端性能时, 测试图片建议统一 `resize` 到 640×640 后再测试。先在 PC 端传输脚本, 再在开发板端执行:

```
# PC 端执行
scp -r tools/gen_pictures.sh eswin@<开发板 IP>:/home/eswin/

# 开发板端执行
chmod a+x gen_pictures.sh
./gen_pictures.sh pictures/input0/000000006818.jpg 640 640
```

运行完成后，基于生成的图片路径进行性能分析。测试前建议将 `postprocess.json` 中的 `score_threshold` 修改为 `0.3`，`iou_thres` 修改为 `0.45`：

```
./sample_npu -s 8 -m quantized_model/ -i 640x640_val2017/ -S '640 640'
```

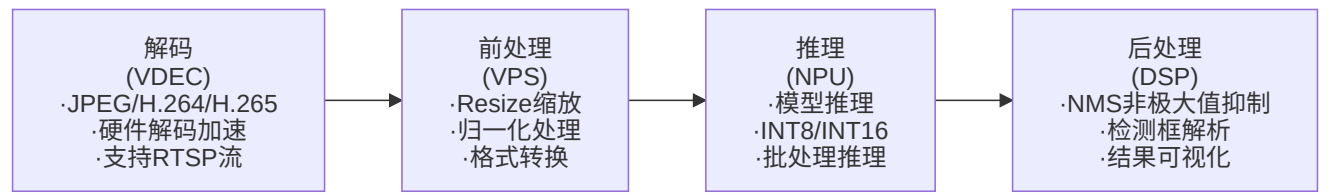
3. 模型运行

本章介绍如何使用 ESSDK 进行模型推理运行，涵盖 Python 和 C++两种编程接口，详细说明解码、前处理、推理、后处理等各环节的硬件加速使用方法，API 具体说明详解各模块开发文档。

3.1. 模型运行概述

3.1.1. 算法运行基本流程

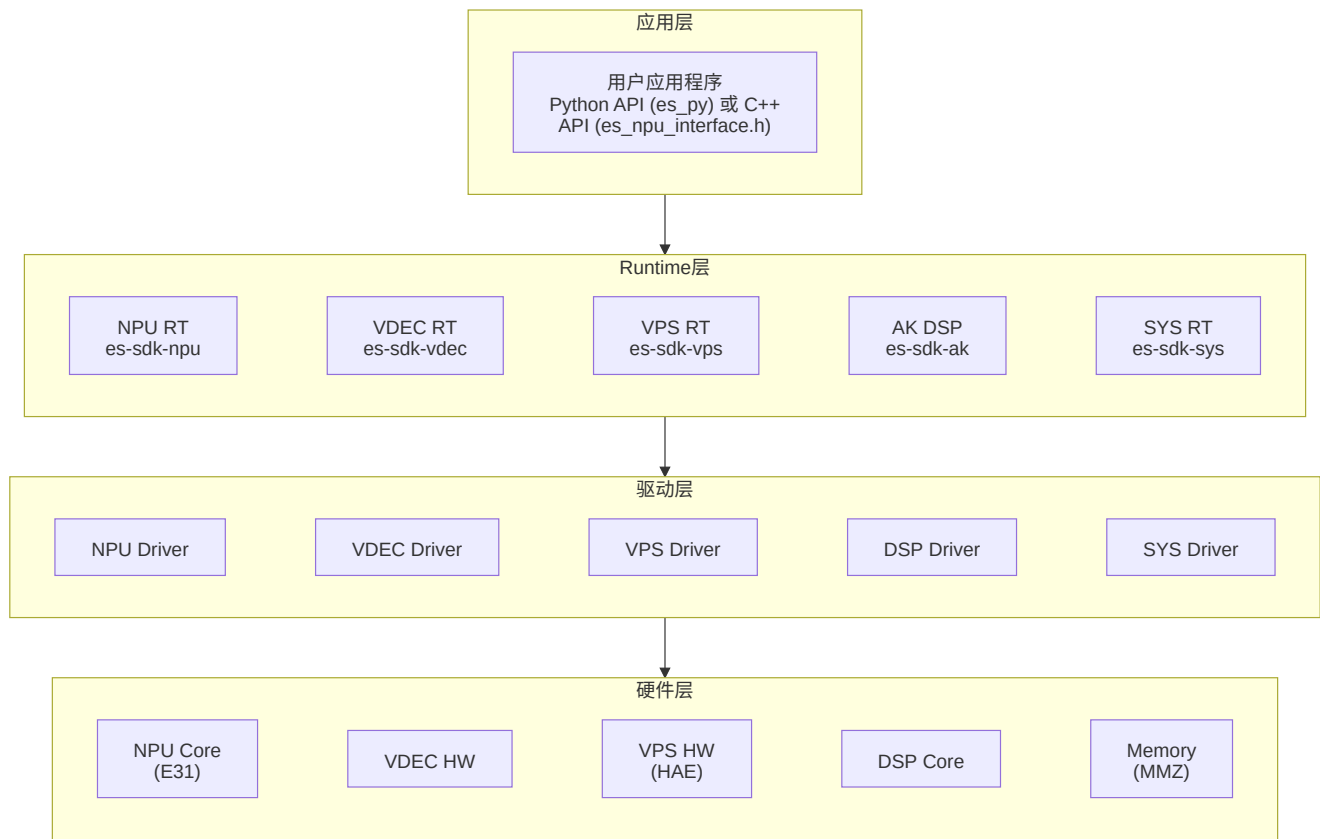
在 EIC7700 平台上，典型的图像/视频类 AI 算法运行流程并非单纯的 NPU 推理，而是一个多硬件单元协作的异构计算流水线。



3.1.2. 硬件加速支持情况

模块	硬件单元	支持功能
解码	VDEC	JPEG/H.264/H.265 硬件解码
前处理	VPS	Resize、归一化、格式转换
推理	NPU	神经网络推理加速
后处理	DSP	NMS、检测框解析

3.1.3. 软硬件模块关系图



3.1.4. 核心数据流

ESSDK 的核心优势在于零拷贝 (Zero-Copy) 数据流。各模块间通过文件描述符 (fd) 共享位于 MMZ (Media Memory Zone) 中的物理内存，避免了 CPU 在模块间搬运大量图像/张量数据。

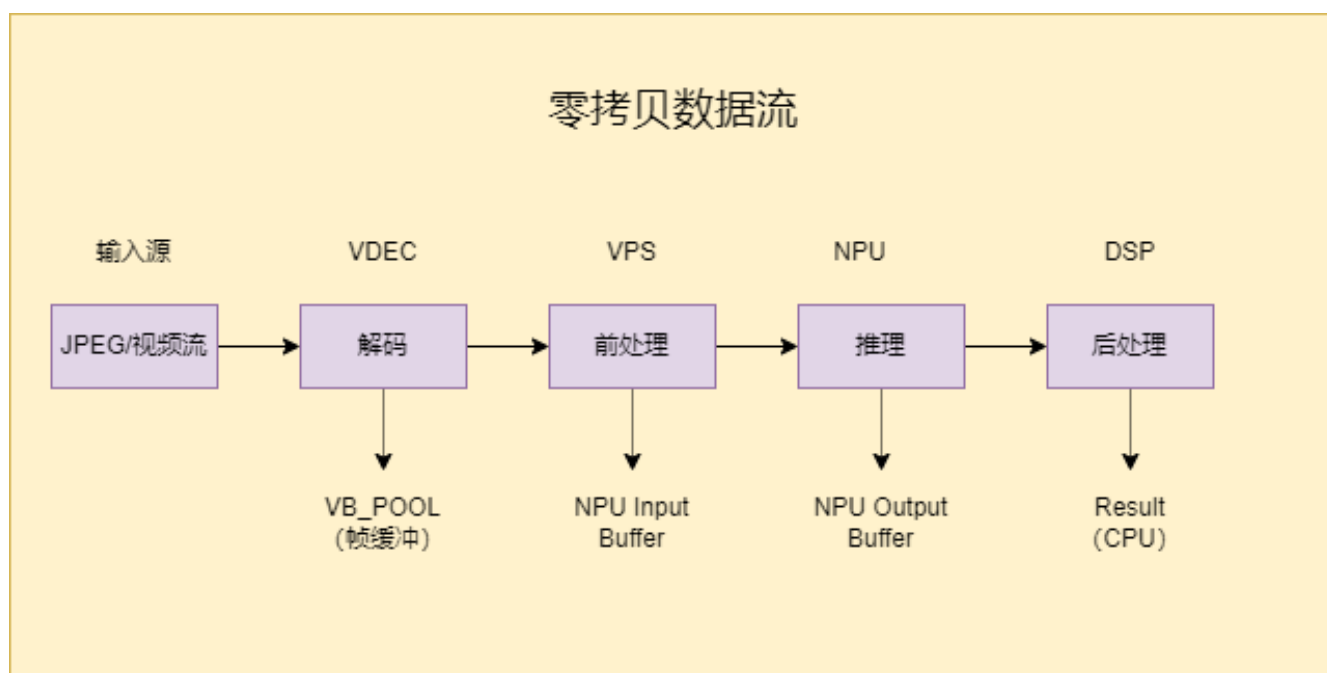


图 10: 数据流

3.2. Python 编程详解

ESWIN 提供 Python SDK (`es_py`)，支持快速开发和原型验证。本节详细介绍 Python 接口的使用方法。

3.2.1. 加载模型

3.2.1.1. 基本流程

```

from es_py import npu, common as c_common

# 1. 查询 NPU 设备数量
num_devices = npu.get_num_devices()
print(f"检测到 {num_devices} 个 NPU 设备")

# 2. 设置当前设备
device_id = 0
rc = npu.set_device(device_id)
if rc != 0:
    raise RuntimeError(f"设置设备失败: rc=0x{rc:08x}")

# 3. 加载模型
model_path = "/path/to/model.model"
model_id = npu.load_model_from_file(model_path)
print(f"模型加载成功: model_id={model_id}")

# 4. 查询模型信息
num_inputs = npu.get_num_input_tensors(model_id)
num_outputs = npu.get_num_output_tensors(model_id)

for i in range(num_inputs):
    desc = npu.get_input_tensor_desc(model_id, i)
    shape = (desc.dims.n, desc.dims.c, desc.dims.h, desc.dims.w)
  
```

```
print(f"输入 {i}: shape={shape}, size={desc.bufferSize}")

for i in range(num_outputs):
    desc = npu.get_output_tensor_desc(model_id, i)
    shape = (desc.dims.n, desc.dims.c, desc.dims.h, desc.dims.w)
    print(f"输出 {i}: shape={shape}, size={desc.bufferSize}")
```

3.2.1.2. 创建 Context 和 Stream

```
# 5. 创建 Context (设备上下文)
context = npu.create_context(device_id)

# 6. 设置当前 Context
npu.set_current_context(context)

# 7. 创建 Stream (计算流)
stream = npu.create_stream()
```

3.2.1.3. 内存分配

```
from es_py import memory

# 分配 NPU 内存
mmz_name = f"mmz_nid_{device_id}_part_0"
fd = memory.memory_alloc(
    c_common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_NOCACHE,
    "buffer_name",
    mmz_name,
    buffer_size
)

# 映射内存到用户空间
memview, capsule = memory.memory_map(
    fd,
    buffer_size,
    c_common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_NOCACHE
)

# 写入数据
memview[0:len(data)] = data

# 刷新缓存
memory.memory_flush_cache(fd)

# 释放内存
memory.memory_free(fd)
```

3.2.2. 解码

ESWIN 提供硬件解码模块 (`es_py.vdec`), 支持 JPEG、H.264、H.265 格式。

3.2.2.1. 初始化 VDEC 模块

```
from es_py import vdec, common as c_common
```

```
# 初始化 VDEC 模块（全局只需执行一次）
vdec.init()
```

3.2.2.2. 创建解码通道

```
# 分配通道 ID
channel_id = 0 # 或使用分配机制获取

# 配置通道属性
attr = c_common.VDEC_GRP_ATTR_S()
attr.type = c_common.PAYLOAD_TYPE_E.PT_JPEG # 或 PT_H264, PT_H265
attr.mode = c_common.VIDEO_MODE_E.VIDEO_MODE_FRAME
attr.picWidth = 1920
attr.picHeight = 1080
attr.frameBufCnt = 1
attr.align = 64

# 计算帧缓冲大小
bufSize = c_common.VDEC_GetPicBufferSize(
    c_common.PIXEL_FORMAT_E.PIXEL_FORMAT_NV12,
    attr.picWidth,
    attr.picHeight,
    attr.align
)
attr.frameBufSize = bufSize

# 创建解码组
die_id = 0
vdec.create_group(channel_id, die_id, attr)

# 配置输出通道
channel_modes = [c_common.VDEC_CHN_MODE_S()]
channel_modes[0].pixelFormat = c_common.PIXEL_FORMAT_E.PIXEL_FORMAT_NV12
channel_modes[0].alpha = 0
channel_modes[0].colorGamut = c_common.COLOR_GAMUT_E.COLOR_GAMUT_BT601

# 使能输出通道
vdec.enable_channel(channel_id, 0)
vdec.set_channel_mode(channel_id, 0, channel_modes[0])

# 开始接收流
vdec.start_receiving_stream(channel_id)
```

3.2.2.3. 发送数据和解码

```
# 读取 JPEG 文件
with open("image.jpg", "rb") as f:
    jpeg_data = f.read()

# 发送数据到解码器
vdec.send_stream(
    channel_id, # 通道 ID
    jpeg_data, # 数据
```

```
True,      # 按帧发送
True,      # 需要解码
False,     # 非结束标志
0,         # 时间戳
-1         # 超时 (-1 表示阻塞)
)

# 获取解码后的帧
rc, frame = vdec.get_frame(channel_id, 0, timeout_ms=1000)
if rc == 0 and frame is not None:
    print(f"解码成功: {frame.videoFrame.width}x{frame.videoFrame.height}")
    # 使用帧数据...

# 释放帧 (必须执行)
vdec.release_frame(channel_id, 0, frame)
```

3.2.2.4. 支持的解码格式

格式	Payload Type	说明
JPEG	PT_JPEG	图片格式，支持任意分辨率
H.264	PT_H264	视频流，需要 Annex B 格式
H.265	PT_H265	视频流，需要 Annex B 格式

提示

MP4/AVI 等容器格式需要先用 FFmpeg 提取裸流：

```
ffmpeg -i input.mp4 -c:v copy -bsf:v h264_mp4toannexb output.h264
```

3.2.3. 前处理

ESWIN 提供 VPS 硬件前处理模块 (`es_py.vps`)，支持 Resize、归一化、格式转换等操作。

3.2.3.1. 初始化 VPS 模块

```
from es_py import vps

# 初始化 VPS 模块 (全局只需执行一次)
vps.init()
```

3.2.3.2. 配置前处理参数

```
# 归一化参数配置
norm_params = c_common.VPS_NORMALIZATION_PARAMS_S()
norm_params.normalizationMode =
↳ c_common.VPS_NORMALIZATION_MODE_E.VPS_NORMALIZATION_Z_SCORE
```



```

# 设置均值 (BGR 顺序)
norm_params.meanValue.b = float_to_u32(0.0)
norm_params.meanValue.g = float_to_u32(0.0)
norm_params.meanValue.r = float_to_u32(0.0)

# 设置标准差倒数 (BGR 顺序)
norm_params.stdReciprocal.b = float_to_u32(1.0 / 0.007874)
norm_params.stdReciprocal.g = float_to_u32(1.0 / 0.007874)
norm_params.stdReciprocal.r = float_to_u32(1.0 / 0.007874)

# 量化步长倒数
norm_params.stepReciprocal = float_to_u32(1.0 / 0.007874)

def float_to_u32(value: float) -> int:
    """将 float 转换为 uint32 用于 VPS 参数传递"""
    import struct
    return struct.unpack('<I', struct.pack('<f', value))[0]

```

3.2.3.3. 执行前处理

```

# 设置 Resize 方式
prop = c_common.VPS_PROPERTY_S()
prop.value = c_common.VPS_RESIZE_METHOD_E.VPS_RESIZE_BILINEAR
vps.set_property(prop)

# 源区域 (整帧)
src_rect = c_common.RECT_S()
src_rect.x = 0
src_rect.y = 0
src_rect.width = src_frame.videoFrame.width
src_rect.height = src_frame.videoFrame.height

# 目标区域 (根据 keep_ratio 计算)
dst_rect = c_common.RECT_S()
dst_rect.x = 0
dst_rect.y = 0
dst_rect.width = target_width
dst_rect.height = target_height

# 执行 Resize
vps.crop_resize(
    src_frame.videoFrame, # 输入帧
    midd_frame.videoFrame, # 中间帧
    src_rect,             # 源区域
    dst_rect,             # 目标区域
    0,                    # 保留参数
    c_common.HWTYPE_E.HW_TYPE_HAE # 硬件类型
)

# 执行归一化
vps.normalization(
    midd_frame.videoFrame, # 输入帧
    dst_frame.videoFrame, # 输出帧
    norm_params,           # 归一化参数
    src_rect,              # 源区域
    dst_rect               # 目标区域
)

```

3.2.3.4. 前处理配置文件示例

```
{
  "resize_shape": [640, 640],
  "mean": [0, 0, 0],
  "std": [1, 1, 1],
  "norm": true,
  "scale": 0.007874,
  "keep_ratio": true,
  "channel_first": true,
  "input_format": "RGB",
  "input_data_type": "S8"
}
```

3.2.4. 推理

3.2.4.1. 构建推理任务

```
# 构建任务
task = c_common.NPU_TASK_S()
task.modelId = model_id

# 设置输入缓冲区
input_buf = input_buffers[0] # 从缓冲池获取
dev_buf = c_common.ES_DEV_BUF_S()
dev_buf.memFd = input_buf.fd
dev_buf.size = input_buf.size
dev_buf.offset = 0
task.set_inputFd(0, dev_buf)
task.inputFdNum = 1

# 设置输出缓冲区
output_buf = output_buffers[0] # 从缓冲池获取
dev_buf = c_common.ES_DEV_BUF_S()
dev_buf.memFd = output_buf.fd
dev_buf.size = output_buf.size
dev_buf.offset = 0
task.set_outputFd(0, dev_buf)
task.outputFdNum = 1
```

3.2.4.2. 同步推理

```
# 提交同步任务（阻塞直到完成）
rc = npu.submit([task])
if rc != 0:
    raise RuntimeError(f"推理失败: rc=0x{rc:08x}")

# 读取输出
output_data = output_buf.read(output_buf.size)
```

3.2.4.3. 异步推理

```

# 定义回调函数
def task_callback(task_obj) -> int:
    # 读取输出数据
    output_data = output_buf.read(output_buf.size)

    # 执行后处理...

    # 标记任务完成
    return 0 # ES_SUCCESS

# 注册回调
task.set_callback_py(task_callback)

# 提交异步任务
rc = npu.submit_async([task], stream)
if rc != 0:
    raise RuntimeError(f"异步提交失败: rc=0x{rc:08x}")

# 查询任务状态（需要在独立线程中执行）
while running:
    rc = npu.process_report(stream, query_ms=100)
    if rc != 0:
        break

```

3.2.4.4. 多流并发推理

```

# 创建多个 Stream
streams = []
for i in range(num_streams):
    stream = npu.create_stream()
    streams.append(stream)

# 向不同 Stream 提交任务
for i, stream in enumerate(streams):
    task = create_task(i) # 创建任务
    npu.submit_async([task], stream)

```

3.2.5. 后处理

ESWIN 提供 DSP 硬件后处理模块 (`es_py.ak`), 支持 YOLO 系列模型的 NMS 等操作。

3.2.5.1. 初始化 AK 模块

```

from es_py import ak

# 初始化 AK 设备
ret = ak.init()
if ret != 0:
    raise RuntimeError(f"AK 初始化失败: ret=0x{ret:08x}")

# 设置 DSP 设备
device = c_common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0
ret = ak.set_device([device])

```

3.2.5.2. 配置检测参数

```
# 创建检测输出配置
cfg = c_common.ES_DETECTION_OUT_CFG()
cfg.clsNum = 80 # 类别数
cfg.scoreThreshold = 0.5 # 置信度阈值
cfg.iouThreshold = 0.45 # IoU 阈值
cfg.maxBoxesPerClass = 50
cfg.maxBoxesPerBatch = 100
cfg.imgH = 640
cfg.imgW = 640
cfg.anchorsNum = 3

# 设置 NMS 方式
cfg.nmsMethod = c_common.ES_NMS_METHOD_E.HARD_NMS
cfg.iouMethod = c_common.ES_IOU_METHOD_E.IOU
cfg.outBoxType = c_common.ES_BOX_TYPE_E.XminYminXmaxYmax

# 设置 anchor scale
anchor_array = cfg.anchorScale
anchors = [10, 13, 16, 30, 33, 23, 30, 61, 62, 45, 59, 119, 116, 90, 156, 198, 373, 326]
for i, val in enumerate(anchors):
    anchor_array[i] = float(val)
```

3.2.5.3. 执行硬件后处理

```
# 准备输出 Tensor
output_tensors = [...] # NPU 输出转换为 ES_TENSOR_S 列表
output_result = [...] # 结果 Tensor
output_count = [...] # 计数 Tensor

# 调用 DSP 后处理
ak.dsp_detection_out(
    device, # DSP 设备
    output_tensors, # 输入 Tensor
    output_result, # 输出结果 Tensor
    output_count, # 输出计数 Tensor
    c_common.ES_DET_NETWORK_E.yolov5, # 网络类型
    cfg # 配置参数
)
```

3.2.5.4. 支持的网络类型

网络类型	枚举值	说明
YOLOv3	yolov3_u, yolov3_d	YOLOv3 系列
YOLOv4	yolov4	YOLOv4
YOLOv5	yolov5	YOLOv5
YOLOv7	yolov7	YOLOv7
YOLOv8	yolov8	YOLOv8
YOLOv11	yolov11	YOLOv11

3.2.5.5. 后处理配置文件示例

```
{
  "detection_net": "yolov5",
  "cls_num": 80,
  "score_threshold": 0.5,
  "iou_threshold": 0.45,
  "max_boxes_per_class": 50,
  "max_boxes_per_batch": 100,
  "img_h": 640,
  "img_w": 640,
  "anchors_num": 3,
  "anchor_scale_table": [[10, 13], [16, 30], [33, 23]],
  "input_scale": [80, 40, 20],
  "input_data_type": "S8,S8,S8"
}
```

3.2.6. 完整示例（多流流水线）

以下示例展示了完整的多流视频检测流水线：

```
"""
YOLO 多流检测流水线示例
支持多路视频流输入，硬件加速的全链路处理
"""
import threading
import queue
import time
from es_py import npu, vdec, vps, ak, memory, common as c_common

class MultiStreamPipeline:
    def __init__(self, config):
        self.config = config
        self.num_streams = len(config.input_sources)

        # 队列系统
        self.frame_queues = [queue.Queue(maxsize=10) for _ in range(self.num_streams)]
        self.batch_queue = queue.Queue(maxsize=50)
        self.result_queue = queue.Queue(maxsize=50)

        # 控制事件
        self.stop_event = threading.Event()

        # 组件初始化
        self._init_system()
        self._init_components()

    def _init_system(self):
        """初始化系统模块"""
        from es_py import sys as es_sys
        es_sys.init()
        vdec.init()
        vps.init()
        ak.init()

    def _init_components(self):
        """初始化各组件"""
        # 初始化 NPU 引擎
        npu.set_device(self.config.device_id)
```

```
self.model_id = npu.load_model_from_file(self.config.model_path)
self.context = npu.create_context(self.config.device_id)
npu.set_current_context(self.context)
self.stream = npu.create_stream()

# 初始化其他组件...

def start(self):
    """启动流水线"""
    # 启动解码线程
    for stream_id in range(self.num_streams):
        thread = threading.Thread(
            target=self._decoder_worker,
            args=(stream_id,),
            daemon=True
        )
        thread.start()

    # 启动前处理线程
    threading.Thread(target=self._preprocess_worker, daemon=True).start()

    # 启动推理线程
    threading.Thread(target=self._inference_worker, daemon=True).start()

    # 启动后处理线程
    threading.Thread(target=self._postprocess_worker, daemon=True).start()

def _decoder_worker(self, stream_id):
    """解码线程"""
    # 解码逻辑...
    pass

def _preprocess_worker(self):
    """前处理线程"""
    # 前处理逻辑...
    pass

def _inference_worker(self):
    """推理线程"""
    # 推理逻辑...
    pass

def _postprocess_worker(self):
    """后处理线程"""
    # 后处理逻辑...
    pass

# 使用示例
if __name__ == "__main__":
    config = {
        "model_path": "/path/to/yolov5.model",
        "input_sources": ["video1.mp4", "video2.mp4"],
        "device_id": 0,
        "batch_size": 2
    }

    pipeline = MultiStreamPipeline(config)
    pipeline.start()

    # 等待处理完成
    pipeline.wait()
```

```
pipeline.stop()
```

3.3. C++编程详解

ESWIN 提供 C++ SDK，适用于高性能部署场景。本节详细介绍 C++接口的使用方法。

3.3.1. 加载模型

3.3.1.1. 头文件引用

```
#include "es_npu_interface.h"
#include "es_sys_memory.h"
#include <iostream>
#include <vector>
```

3.3.1.2. 基本流程

```
ES_S32 ret;
ES_U16 deviceId = 0;
ES_U32 modelId;

//1. 查询 NPU 设备数量
ES_U16 deviceNum = 0;
ret = ES_NPU_GetNumDevices(&deviceNum);
if (ret != ES_SUCCESS) {
    printf("ES_NPU_GetNumDevices failed, ret: 0x%x\n", ret);
    return -1;
}
printf("检测到 %d 个 NPU 设备\n", deviceNum);

//2. 设置设备
ret = ES_NPU_SetDevice(deviceId);
if (ret != ES_SUCCESS) {
    printf("ES_NPU_SetDevice failed, ret: 0x%x\n", ret);
    return -1;
}

//3. 加载模型
const char* modelPath = "/path/to/model.model";
ret = ES_NPU_LoadModelFromFile(&modelId, modelPath);
if (ret != ES_SUCCESS) {
    printf("ES_NPU_LoadModelFromFile failed, ret: 0x%x\n", ret);
    return -1;
}
printf("模型加载成功: modelId=%d\n", modelId);

//4. 查询模型输入输出信息
ES_S32 numInputs, numOutputs;
ret = ES_NPU_GetNumInputTensors(modelId, &numInputs);
ret = ES_NPU_GetNumOutputTensors(modelId, &numOutputs);

for (ES_S32 i = 0; i < numInputs; i++) {
    NPU_TENSOR_S tensor;
    ret = ES_NPU_GetInputTensorDesc(modelId, i, &tensor);
    printf("输入 %d: shape=[%d,%d,%d,%d], size=%d\n",
```

```

        i, tensor.dims.n, tensor.dims.c, tensor.dims.h, tensor.dims.w,
        tensor.bufferSize);
    }

```

3.3.1.3. 创建 Context 和 Stream

```

npu_context context;
npu_stream stream;

// 创建 Context
ret = ES_NPU_CreateContext(&context, deviceId);
if (ret != ES_SUCCESS) {
    printf("ES_NPU_CreateContext failed, ret: 0x%x\n", ret);
    return -1;
}

// 设置当前 Context
ret = ES_NPU_SetCurrentContext(context);

// 创建 Stream
ret = ES_NPU_CreateStream(&stream);
if (ret != ES_SUCCESS) {
    printf("ES_NPU_CreateStream failed, ret: 0x%x\n", ret);
    ES_NPU_DestroyContext(context);
    return -1;
}

```

3.3.2. 解码

C++解码通过 `IVdec` 接口类实现，支持 JPEG、H.264、H.265 硬件解码。

3.3.2.1. 解码器接口

```

// IVdec 接口定义
class IVdec {
public:
    virtual ~IVdec() = default;

    // 初始化解码器
    virtual ES_S32 initVdec(ES_U16 deviceNum) = 0;

    // 解码 JPEG
    virtual ES_S32 decodeJpeg(const JPEG_DATA& jpegData) = 0;

    // 获取解码帧
    virtual bool getVideoFrame(VDEC_VIDEO_FRAME& frame) = 0;

    // 释放帧
    virtual void releaseFrame(const VDEC_VIDEO_FRAME& frame) = 0;
};

```

3.3.2.2. 使用示例


```

#include "EsVdec.h"

// 创建解码器实例
std::shared_ptr<IVdec> vdec = std::make_shared<EsVdec>(deviceId);

// 初始化
ret = vdec->initVdec(deviceNum);
if (ret != ES_SUCCESS) {
    printf("初始化解码器失败\n");
    return -1;
}

// 读取 JPEG 文件
std::vector<char> fileData;
ret = read_jpeg_data("image.jpg", fileData);

// 解码
JPEG_DATA jpegData;
jpegData.data = reinterpret_cast<uint8_t*>(fileData.data());
jpegData.len = static_cast<uint32_t>(fileData.size());

ret = vdec->decodeJpeg(jpegData);
if (ret != ES_SUCCESS) {
    printf("解码失败\n");
    return -1;
}

// 获取解码帧
VDEC_VIDEO_FRAME frame;
if (vdec->getVideoFrame(frame)) {
    // 使用帧数据进行前处理
    // ...

    // 释放帧
    vdec->releaseFrame(frame);
}

```

3.3.3. 前处理

C++前处理通过 `IPreprocess` 接口类实现，使用 VPS 硬件加速。

3.3.3.1. 前处理接口

```

// IPreprocess 接口定义
class IPreprocess {
public:
    virtual ~IPreprocess() = default;

    // 初始化配置
    virtual bool initializeConfig(ES_U16 deviceId, const PreprocessConfig& config) = 0;

    // 执行前处理
    virtual ES_S32 preprocess(const VIDEO_FRAME_INFO_S& srcFrame,
                             PREPROCESSED_DATA& processedData) = 0;

    // 更新输出精度
    virtual void updateOutputPrecision(ES_DATA_PRECISION_E precision) = 0;
};

```

```
// 前处理数据结构
struct PREPROCESSED_DATA {
    uint64_t fd;    // 输出缓冲区 fd
    uint64_t offset; // 偏移量
    uint64_t size;  // 大小
};
```

3.3.3.2. 使用示例

```
#include "EsHwPreProcess.h"

// 创建前处理器实例
std::shared_ptr<IPreprocess> preprocessor = std::make_shared<EsHwPreProcess>(deviceId);

// 加载配置
ConfigParser configParser;
configParser.loadPreprocessConfig("preprocess.json");
PreprocessConfig config = configParser.getPreprocessConfig();

// 初始化
if (!preprocessor->initializeConfig(deviceId, config)) {
    printf("前处理器初始化失败\n");
    return -1;
}

// 执行前处理
PREPROCESSED_DATA processedData;
processedData.fd = npuInputFd; // NPU 输入缓冲区 fd
processedData.offset = 0;
processedData.size = inputSize;

ret = preprocessor->preprocess(frame.videoInfoFrame, processedData);
if (ret != ES_SUCCESS) {
    printf("前处理失败\n");
    return -1;
}
```

3.3.3.3. 前处理配置结构

```
struct PreprocessConfig {
    std::vector<int> resize_shape; // 目标尺寸
    std::vector<float> mean;      // 均值
    std::vector<float> std;       // 标准差
    bool norm;                   // 是否归一化
    float scale;                 // 缩放因子
    bool keep_ratio;             // 保持宽高比
    bool channel_first;          // 通道优先
    std::string input_format;    // 输入格式 "RGB" 或 "BGR"
    nlohmann::json params;      // 其他参数
};
```

3.3.4. 推理

3.3.4.1. 构建任务

```
// 分配输入输出内存
ES_DEV_BUF_S inputFd[ES_TASK_MAX_FD_CNT];
ES_DEV_BUF_S outputFd[ES_TASK_MAX_FD_CNT];

// 输入内存
for (ES_S32 i = 0; i < numInputs; i++) {
    NPU_TENSOR_S tensor;
    ES_NPU_GetInputTensorDesc(modelId, i, &tensor);

    ES_SYS_MemAlloc(&inputFd[i].memFd,
        SYS_CACHE_MODE_NOCACHE,
        "npu_input",
        "mmz_nid_0_part_0",
        tensor.bufferSize);

    inputFd[i].size = tensor.bufferSize;
    inputFd[i].offset = 0;

    ES_NPU_Prepare(&inputFd[i]);
}

// 输出内存
for (ES_S32 i = 0; i < numOutputs; i++) {
    NPU_TENSOR_S tensor;
    ES_NPU_GetOutputTensorDesc(modelId, i, &tensor);

    ES_SYS_MemAlloc(&outputFd[i].memFd,
        SYS_CACHE_MODE_NOCACHE,
        "npu_output",
        "mmz_nid_0_part_0",
        tensor.bufferSize);

    outputFd[i].size = tensor.bufferSize;
    outputFd[i].offset = 0;

    ES_NPU_Prepare(&outputFd[i]);
}

// 构建任务
NPU_TASK_S task;
memset(&task, 0, sizeof(NPU_TASK_S));
task.modelId = modelId;
task.taskId = taskId++;
task.inputFdNum = numInputs;
task.outputFdNum = numOutputs;
memcpy(task.inputFd, inputFd, sizeof(inputFd));
memcpy(task.outputFd, outputFd, sizeof(outputFd));
```

3.3.4.2. 同步推理

```
// 提交同步任务
ret = ES_NPU_Submit(&task, 1);
if (ret != ES_SUCCESS) {
    printf("ES_NPU_Submit failed, ret: 0x%x\n", ret);
}
```

```

    return -1;
}

// 任务完成后读取输出
// ...

```

3.3.4.3. 异步推理

```

// 定义回调函数
ES_S32 taskCallback(ES_VOID* args) {
    SampleCallbackArgs* callbackArgs = (SampleCallbackArgs*)args;

    // 获取输出 Tensor
    std::vector<ES_TENSOR_S> outputTensors;
    getOutputTensors(*callbackArgs->task, false, outputTensors);

    // 执行后处理
    InferenceOutput output;
    callbackArgs->postProcess->run(outputTensors, output, callbackArgs->pictures);

    // 清理资源
    destroyTask(*callbackArgs->task);
    delete callbackArgs->task;
    delete callbackArgs;

    return ES_SUCCESS;
}

// 设置回调
SampleCallbackArgs* callbackArgs = new SampleCallbackArgs;
callbackArgs->task = new NPU_TASK_S(task);
callbackArgs->postProcess = postProcess;
task.callback = taskCallback;
task.callbackArg = callbackArgs;

// 提交异步任务
ret = ES_NPU_SubmitAsync(&task, 1, stream);
if (ret != ES_SUCCESS) {
    printf("ES_NPU_SubmitAsync failed, ret: 0x%x\n", ret);
    return -1;
}

// 查询线程 (需要在独立线程中运行)
void* queryThread(void* args) {
    SampleStateInfo* info = (SampleStateInfo*)args;
    while (!info->queryThreadExited) {
        ES_S32 ret = ES_NPU_ProcessReport(info->stream, info->queryMs);
        if (ret != ES_SUCCESS) {
            break;
        }
    }
    return nullptr;
}

```

3.3.4.4. 多流并发

```

// 创建多个 Stream
std::vector<npu_stream> streams;
for (int i = 0; i < numStreams; i++) {
    npu_stream stream;
    ES_NPU_CreateStream(&stream);
    streams.push_back(stream);
}

// 向不同 Stream 提交任务
for (int i = 0; i < numStreams; i++) {
    NPU_TASK_S task = createTask(i);
    ES_NPU_SubmitAsync(&task, 1, streams[i]);
}

// 等待所有任务完成
for (auto& stream : streams) {
    ES_NPU_SynchronizeStream(stream);
}

```

3.3.5. 后处理

C++后处理通过 `Postprocess` 类实现，支持 DSP 硬件加速和 CPU NMS。

3.3.5.1. 后处理接口

```

class Postprocess {
public:
    // 创建后处理算子
    ES_S32 createOps(const std::vector<nlohmann::json>& configs, ES_U16 deviceId);

    // 执行后处理
    void run(const std::vector<ES_TENSOR_S>& tensors,
            InferenceOutput& output,
            const std::vector<std::string>& pictures);
};

```

3.3.5.2. 使用示例

```

#include "Postprocess.h"

// 创建后处理器
std::shared_ptr<Postprocess> postProcess = std::make_shared<Postprocess>();

// 加载配置
ConfigParser configParser;
configParser.loadPostprocessConfig("postprocess.json");
configParser.loadClassesConfig("classes.txt");

// 创建后处理算子
ret = postProcess->createOps(configParser.getPostprocessConfigs(), deviceId);
if (ret != ES_SUCCESS) {
    printf("创建后处理算子失败\n");
    return -1;
}

```

```

// 执行后处理
InferenceOutput output;
postProcess->run(outputTensors, output, pictures);

// 输出结果
for (const auto& det : output.detections) {
    printf("类别: %s, 置信度: %.2f, 边框: [%f, %f, %f, %f]\n",
        det.className.c_str(), det.confidence,
        det.box[0], det.box[1], det.box[2], det.box[3]);
}

```

3.3.5.3. 检测结果结构

```

struct Detection {
    int classId;           // 类别 ID
    std::string className; // 类别名称
    float confidence;      // 置信度
    std::vector<float> box; // 边框 [x1, y1, x2, y2]
};

struct InferenceOutput {
    std::vector<Detection> detections;
    float processTime;
};

```

3.3.6. 完整示例

3.3.6.1. 同步推理示例

```

ES_S32 SAMPLE_NPU_Sync(ES_U16 devId, ES_U32 modelId,
    std::vector<std::vector<std::string>>& inputFiles,
    const std::string& outputDir,
    std::shared_ptr<IVdec> vdec,
    std::shared_ptr<IPreprocess> preProcess,
    std::shared_ptr<Postprocess> postProcess)
{
    ES_S32 ret = ES_SUCCESS;
    ES_U32 taskId = 0;

    while (!inputFiles.front().empty()) {
        NPU_TASK_S task;
        memset(&task, 0, sizeof(NPU_TASK_S));
        std::vector<std::vector<std::string>> pictures;

        // 构建任务 (包含解码、前处理)
        ret = SAMPLE_NPU_COMM_ConstructTask(devId, modelId, inputFiles,
            vdec, preProcess, task, pictures);
        if (ret != ES_SUCCESS) {
            printf("ConstructTask failed\n");
            return ret;
        }

        task.modelId = modelId;
        task.taskId = taskId++;
    }
}

```

```

// 提交同步推理
ret = ES_NPU_Submit(&task, 1);
if (ret != ES_SUCCESS) {
    printf("ES_NPU_Submit failed\n");
    SAMPLE_NPU_COMM_DestroyTask(task);
    return ES_FAILURE;
}

// 获取输出并执行后处理
std::vector<ES_TENSOR_S> outputTensors;
getOutputTensors(task, false, outputTensors);

InferenceOutput output;
postProcess->run(outputTensors, output, pictures.front());

SAMPLE_NPU_COMM_DestroyTask(task);
}

return ES_SUCCESS;
}

```

3.3.6.2. 异步推理示例

```

ES_S32 SAMPLE_NPU_ASync(ES_U16 devId, ES_U32 modelId, npu_stream stream,
    std::vector<std::vector<std::string>>& inputFiles,
    const std::string& outputDir,
    std::shared_ptr<IVdec> vdec,
    std::shared_ptr<IPreprocess> preProcess,
    std::shared_ptr<Postprocess> postProcess)
{
    ES_S32 ret = ES_SUCCESS;
    std::atomic<ES_S32> completeNum(0);
    SampleStateInfo sampleInfo;

    // 创建查询线程
    sampleInfo.completeNum = &completeNum;
    SAMPLE_NPU_COMM_CreateQueryTrd(sampleInfo, stream);

    uint64_t taskTotalCount = inputFiles.front().size();

    while (!inputFiles.front().empty()) {
        NPU_TASK_S* task = new NPU_TASK_S;
        memset(task, 0, sizeof(NPU_TASK_S));

        SampleCallbackArgs* callbackArgs = new SampleCallbackArgs;
        callbackArgs->completeNum = &completeNum;
        callbackArgs->postProcess = postProcess;
        callbackArgs->task = task;
        callbackArgs->totalTask = taskTotalCount;

        // 构建任务
        ret = SAMPLE_NPU_COMM_ConstructTask(devId, modelId, inputFiles,
            vdec, preProcess, *task, callbackArgs->pictures);
        if (ret != ES_SUCCESS) {
            delete callbackArgs;
            delete task;
            break;
        }
    }
}

```

```

task->modelId = modelId;
task->taskId = sampleInfo.taskNum++;
task->callback = SAMPLE_NPU_COMM_TaskCallback;
task->callbackArg = callbackArgs;

// 提交异步任务
ret = ES_NPU_SubmitAsync(task, 1, stream);
if (ret != ES_SUCCESS) {
    SAMPLE_NPU_COMM_DestroyTask(*task);
    delete callbackArgs;
    delete task;
    break;
}
}

// 等待所有任务完成
while (completeNum.load() != sampleInfo.taskNum.load()) {
    sleep(1);
}

// 停止查询线程
sampleInfo.queryThreadExited = true;
if (sampleInfo.queryThread.joinable()) {
    sampleInfo.queryThread.join();
}

return ret;
}

```

3.3.6.3. 多流示例

```

ES_S32 SAMPLE_NPU_MultiStream(ES_U16 devId, ES_S32 nums, ES_U32 modelId,
    std::vector<std::vector<std::string>>& inputFiles,
    const std::string& outputDir,
    std::shared_ptr<IVdec> vdec,
    std::shared_ptr<IPreprocess> preProcess,
    std::shared_ptr<Postprocess> postProcess)
{
    ES_S32 ret;
    npu_context context;
    std::vector<npu_stream> streamVec;

    // 创建 Context
    ret = ES_NPU_CreateContext(&context, devId);
    if (ret != ES_SUCCESS) {
        return ret;
    }

    // 创建多个 Stream
    for (ES_S32 i = 0; i < nums; i++) {
        npu_stream stream;
        ret = ES_NPU_CreateStream(&stream);
        if (ret != ES_SUCCESS) {
            goto destroy_stream;
        }
        streamVec.push_back(stream);
    }
}

```



```

// 向每个 Stream 提交任务
for (auto& stream : streamVec) {
    ret = SAMPLE_NPU_COMM_SubmitAsync(devId, modelId, stream, inputFiles,
                                     outputDir, vdec, preProcess, postProcess);
    if (ret != ES_SUCCESS) {
        printf("SubmitAsync failed\n");
    }
}

destroy_stream:
// 销毁所有 Stream
for (auto& stream : streamVec) {
    ES_NPU_DestroyStream(stream);
}

// 销毁 Context
ES_NPU_DestroyContext(context);

return ret;
}

```

3.3.6.4. D2D（双 Die）示例

```

ES_S32 SAMPLE_NPU_D2D(std::vector<std::vector<std::string>>& inputFiles,
                     const std::string& outputDir,
                     const std::string& modelPath,
                     std::map<ES_U16, std::shared_ptr<IVdec>>& vdec,
                     std::map<ES_U16, std::shared_ptr<IPreprocess>>& preProcess,
                     std::map<ES_U16, std::shared_ptr<Postprocess>>& postProcess,
                     std::map<ES_U16, std::shared_ptr<ConfigParser>>& configParser)
{
    ES_U16 deviceNum = 0;
    std::vector<std::thread> threadVec;
    std::mutex mtx;

    // 获取设备数量
    ES_NPU_GetNumDevices(&deviceNum);

    // 每个 Die 启动一个处理线程
    for (ES_U16 die = 0; die < deviceNum; die++) {
        threadVec.emplace_back(std::thread(SAMPLE_NPU_COMM_D2DProcessTrd,
                                           die, modelPath,
                                           std::ref(inputFiles),
                                           outputDir,
                                           vdec[die], preProcess[die],
                                           postProcess[die], configParser[die],
                                           std::ref(mtx)));
    }

    // 等待所有线程完成
    for (auto& trd : threadVec) {
        if (trd.joinable()) {
            trd.join();
        }
    }

    return ES_SUCCESS;
}

```

```
}
```

3.3.7. 资源释放

```
// 释放任务内存
void SAMPLE_NPU_COMM_DestroyTask(NPU_TASK_S& task) {
    for (ES_S32 i = 0; i < task.inputFdNum; i++) {
        ES_NPU_Unprepare(&task.inputFd[i]);
        ES_SYS_MemFree(task.inputFd[i].memFd);
    }

    for (ES_S32 i = 0; i < task.outputFdNum; i++) {
        ES_NPU_Unprepare(&task.outputFd[i]);
        ES_SYS_MemFree(task.outputFd[i].memFd);
    }
}

// 释放 NPU 资源
void releaseNPUResources() {
    ES_NPU_DestroyStream(stream);
    ES_NPU_DestroyContext(context);
    ES_NPU_UnloadModel(modelId);
    ES_NPU_ReleaseDevice(deviceId);
}

// 释放 VDEC 资源
void releaseVdecResources() {
    SAMPLE_COMM_VPS_Deinit();
    SAMPLE_COMM_VDEC_Stop(deviceNum);
    SAMPLE_COMM_VDEC_ExitVBPool();
    SAMPLE_COMM_VDEC_Deinit();
    ES_SYS_Exit();
}
```

4. 大模型部署与运行 (DeepSeek)

本章介绍如何在 EIC77 开发板上部署并运行 DeepSeek 模型。文中以 `deepseek_7b_1k_int8` 为例说明单 die 场景；双 die 场景请使用带 `_peer` 后缀的对应模型包。

4.1. 获取模型包

从 ESWIN 资料中心（需要注册）下载模型包：

提示

- EIC7700（单 die）选择 **不带 peer** 的包。
- EIC7702（双 die）选择 **带 peer** 的包。

ESWIN 奕斯伟计算

技术 解决方案 支持 开发者社区 投资者天系 公司

资料中心

EIC7700 Single Board Computer(SBC)

HiFive Premier P550

EIEBX10-A30(IWB)

EIEDBM5-310(Z530)

EIC7700FG_EVB_LP5_A2

EIC770X 开发资料

NN Tools

release-20250730

无线连接

内核与工具链

多媒体处理

生态方案

EIC7x_Release_20250730_nn.tar.gz	20.9G	2025-09-18 16:08	▼
modelzoo-vision.tar.gz	4.6G	2025-11-28 17:33	▼
modelzoo-deepseek_7b_1k_int8.tar.gz	6.4G	2025-10-21 15:00	▼
modelzoo-deepseek_7b_1k_int8_peer.tar.gz	6.5G	2025-10-21 15:00	▼

扫码分享：

图 11: deepseek 模型下载

解压后文件示例如下：

```
— modelzoo-deepseek_7b_1k_int8_0730
  — bin
    — es_qwen2
  — models
    — deepseek_7b_1k_int8
  — README.md
  — src
    — build.sh
    — CMakeLists.txt
    — deepseek_7b_1k_int8
    — deepseek_7b_1k_int8_peer
    — deepseek_7b_1k_int8_peer_060
    — es_qwen2_client.cpp
    — es_qwen2_client.hpp
    — es_qwen2_demo.cpp
    — es_qwen2_process.cpp
    — README.md
    — third_party
    — tokenizer
  — utils
    — eic7700-hifive-premier-p550.dtb
    — eic7702-evb-a1.dtb
— modelzoo-deepseek_7b_1k_int8_0730.tar.gz
— modelzoo-deepseek_7b_1k_int8_peer_0730
  — bin
    — es_qwen2
  — models
    — deepseek_7b_1k_int8_peer
  — README.md
  — src
    — build.sh
    — CMakeLists.txt
    — deepseek_7b_1k_int8
    — deepseek_7b_1k_int8_peer
    — deepseek_7b_1k_int8_peer_060
    — es_qwen2_client.cpp
    — es_qwen2_client.hpp
    — es_qwen2_demo.cpp
    — es_qwen2_process.cpp
    — README.md
    — third_party
    — tokenizer
  — utils
    — eic7700-hifive-premier-p550.dtb
    — eic7702-evb-a1.dtb
— modelzoo-deepseek_7b_1k_int8_peer_0730.tar.gz
```

图 12: deepseek 目录

4.2. 版本兼容性确认

部署前请先确认 **DeepSeek 模型包版本与系统基础镜像版本完全匹配**。例如模型包基于 0730 版本制作，则开发板镜像也必须为 0730。

在开发板端查看系统版本：

```
uname -a
```

```
eswin@rockos-eswin:~$ ls -alh
total 24K
drwx----- 3 eswin eswin 4.0K Dec  2 19:09 .
drwxr-xr-x 3 root root 4.0K Sep  4 2025 ..
-rw-r--r-- 1 eswin eswin 220 Sep 16 13:09 .bash_logout
-rw-r--r-- 1 eswin eswin 3.5K Sep 16 13:09 .bashrc
drwx----- 3 eswin eswin 4.0K Dec  2 19:09 .config
-rw-r--r-- 1 eswin eswin 807 Sep 16 13:09 .profile
eswin@rockos-eswin:~$ uname -a
Linux rockos-eswin 6.6.92-eic7x-2025.07 #2025.09.04.09.57+ SMP Thu Sep  4 09:59:13 CST 2025 riscv64 GNU/Linux
eswin@rockos-eswin:~$
```

图 13: 版本查看

4.3. 模型拷贝

将解压后的 DeepSeek 目录拷贝到开发板，例如拷贝到当前用户家目录：

```
scp -r ./deepseek eswin@<board-ip>:~/
```

拷贝完成后，开发板侧可看到类似目录结构：

```
eswin@rockos-eswin:~$ ls -alh deepseek/
total 32K
drwxr-xr-x 6 eswin eswin 4.0K Apr 27 15:32 .
drwx----- 5 eswin eswin 4.0K Apr 27 15:30 ..
drwxr-xr-x 2 eswin eswin 4.0K Apr 27 15:30 bin
drwxr-xr-x 3 eswin eswin 4.0K Apr 27 15:30 models
-rw-r--r-- 1 eswin eswin 5.0K Apr 27 15:32 README.md
drwxr-xr-x 7 eswin eswin 4.0K Apr 27 15:32 src
drwxr-xr-x 2 eswin eswin 4.0K Apr 27 15:32 utils
eswin@rockos-eswin:~$
```

图 14: 目录结构

4.4. 扩充 MMZ 到 11G

DeepSeek 7B 模型对 DDR 和 MMZ (Media Memory Zone) 预留空间要求较高。本文示例统一配置 **11GB MMZ**。

通过 PC 连接板卡串口，在系统启动时进入 U-Boot 界面后执行：

```
setenv fdt_cmd "fdt mmz mmz_nid_0_part_0 0x1c0000000 0x2c0000000"
setenv bootcmd "run fdt_cmd;${bootcmd}"
saveenv
reset
```

如果上述配置错误，可恢复默认环境变量：

```
env default -a
saveenv
```

```
reset
```

设置完成后，进入 Linux 系统，通过以下命令检查 MMZ 是否生效：

```
cat /proc/eswin/vb
```

终端示例输出应显示 MMZ 区域大小约为 11GB：

```
-----MMZ REGION CONFIG-----
  memblock: mmz_nid_0_part_0, total size(0x2bffd000), free mem size(0x2bffd000)
-----POOL CONFIG-----
```

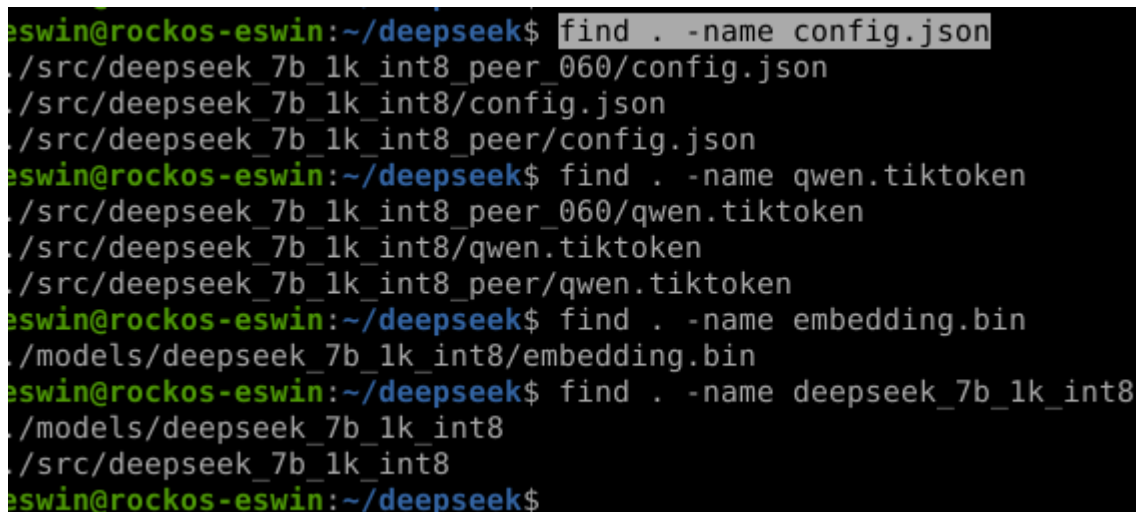
4.5. 修改模型配置文件

先定位模型目录和依赖文件：

```
find . -name config.json
find . -name deepseek_7b_1k_int8
find . -name qwen.tiktoken
find . -name embedding.bin
```

模型选择规则：

- **EIC7700 (单 die)**：使用 `deepseek_7b_1k_int8`
- **EIC7702 (双 die)**：使用带 `_peer` 后缀的模型



```
eswin@rockos-eswin:~/deepseek$ find . -name config.json
./src/deepseek_7b_1k_int8_peer_060/config.json
./src/deepseek_7b_1k_int8/config.json
./src/deepseek_7b_1k_int8_peer/config.json
eswin@rockos-eswin:~/deepseek$ find . -name qwen.tiktoken
./src/deepseek_7b_1k_int8_peer_060/qwen.tiktoken
./src/deepseek_7b_1k_int8/qwen.tiktoken
./src/deepseek_7b_1k_int8_peer/qwen.tiktoken
eswin@rockos-eswin:~/deepseek$ find . -name embedding.bin
./models/deepseek_7b_1k_int8/embedding.bin
eswin@rockos-eswin:~/deepseek$ find . -name deepseek_7b_1k_int8
./models/deepseek_7b_1k_int8
./src/deepseek_7b_1k_int8
eswin@rockos-eswin:~/deepseek$
```

图 15: 查找文件

打开对应模型目录下的 `config.json`，将其中涉及的路径改为**开发板上的绝对路径**。至少需要确认以下内容与实际文件位置一致：

- 模型目录路径
- `qwen.tiktoken` 路径
- `embedding.bin` 路径

```
eswin@rockos-eswin:~/deepseek$ realpath ./models/deepseek_7b_1k_int8
/home/eswin/deepseek/models/deepseek_7b_1k_int8
eswin@rockos-eswin:~/deepseek$ realpath ./src/deepseek_7b_1k_int8/qwen.tiktoken
/home/eswin/deepseek/src/deepseek_7b_1k_int8/qwen.tiktoken
eswin@rockos-eswin:~/deepseek$ realpath ./models/deepseek_7b_1k_int8/embedding.bin
/home/eswin/deepseek/models/deepseek_7b_1k_int8/embedding.bin
eswin@rockos-eswin:~/deepseek$
```

图 16: 实际路径

例如:

```
cd ~/deepseek
vim.tiny ./src/deepseek_7b_1k_int8/config.json
```

修改完成后, 建议用 `grep` 再次确认关键路径已经写入:

```
grep -E 'deepseek_7b_1k_int8|qwen.tiktoken|embedding.bin' ./src/deepseek_7b_1k_int8/config.json
```

4.6. 运行 DeepSeek 模型

DeepSeek 在当前交付方式下属于 **基于 Qwen 底座适配的大模型包**, 因此板端仍通过 `es-sdk-sample-npu-qwen` 提供的 `es_qwen2` 二进制程序完成统一调度。

`es_qwen2` 的职责是:

- 解析 `config.json`
- 加载 tokenizer、embedding 和模型文件
- 调度底层 Runtime 执行推理

因此, **模型是 DeepSeek, 运行入口程序是 `es_qwen2`**。

```
sudo apt update

#安装
sudo apt install -y es-sdk-sample-npu-qwen

#确认路径下应存在可执行文件
ls -alh /opt/eswin/sample-code/npu_sample/qwen_sample/bin/es_qwen2

#运行
cd ~/deepseek
/opt/eswin/sample-code/npu_sample/qwen_sample/bin/es_qwen2 ./src/deepseek_7b_1k_int8/config.json
```

```
eswin@rockos-eswin:~/deepseek$ /opt/eswin/sample-code/npu_sample/qwen_sample/bin/es_qwen2 ./src/deepseek_7b_1k_int8/config.json
Loading models: [=====] 100.00% ( 52.878665 seconds )
-----
0: Role setting: 你是一个智能助理。
-----
1: 介绍一下大语言模型
2: The quantum computers
3: Humans and robots coexist
4: Customized prompts
-----
[YOU]: 1
[YOU]: 介绍一下大语言模型
[Qwen2]: <think>
嗯，用户让我介绍一下大语言模型，也就是 LLM。我得先理解什么是大语言模型。可能是指像 GPT 这样的 AI 模型，对吧？那它有什么特点呢？首先，它应该能处理大量的文本数据，包括文本、图像和音频，这可能让它能更全面地理解信息。

然后，它具备多轮对话的能力，这让我想到在聊天时，它能像人类那样进行自然的交流，可能需要记住之前的对话内容。这点很重要，因为它能提供更连贯的帮助。

生成能力也很关键。比如，当用户输入一个指令或问题时，它能迅速给出答案或建议。这可能涉及到不同的领域，比如数学计算、信息检索或者创造力激发。

用户可能会想，这些模型是怎么工作的？可能涉及到大量的数据和复杂的算法，比如使用像神经 & 转录这样的技术来理解结构化数据，或者自然 & 召遇这样的技术来处理语音指令。

那这些模型的使用场景是什么呢？可能在教育、客服、内容创作等多个领域都有应用。比如，教育机构可以使用它来生成学习资料，客服可以提供更自然的对话体验，而内容创作者则可以快速生成高质量的内容。

不过，用户可能也关心这些模型的局限性。比如，它们可能无法理解复杂的逻辑或伦理问题，或者在处理模糊或误导性信息时表现不佳。此外，用户可能也需要知道这些模型的来源，比如它们是开源的，比如 GPT-? 系列，这可能影响它们的可用性和定制性。

总的来说，大语言模型是一个强大的工具，能够帮助用户生成和理解文本内容，但用户在使用时也需要了解其局限性，以更好地评估和应用这些模型。
```

图 17: 运行界面示例